

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение высшего
образования
«Кузбасский государственный технический университет имени Т.Ф. Горбачева»
Филиал КузГТУ в г. Белово

ОТЧЕТ
ПО УЧЕБНОЙ ПРАКТИКЕ
ПМ.07.01 «Сoadминистрирование баз данных и серверов»
База практики филиал КузГТУ в г.Белово

Выполнил:
Студент группы ИС-235
_____ /Ботев Г. Е.

Руководитель практики от
филиала КузГТУ в г. Белово
_____ / Витвицкий М.Н.

М.П.

Оценка _____
« ____ » _____ 2026 г.

Белово
2026

ЗАДАНИЕ

на учебную практику

по профессионального модуля **ПМ.07 «Сопровождение информационных систем»**

студент Ботев Григорий Евгеньевич

группы ИС-225.2

специальности «09.02.07 Информационные системы и программирование»

Дата начала практики «13» января 2025 г.

Дата окончания практики «05» июня 2026.

Дата сдачи практики «03» июня 2026.

Виды работ, обязательных для выполнения:

1. Рассмотрение обязанностей администратора баз данных. Основные утилиты администратора баз данных. Режимы запуска и останова базы данных.
2. Рассмотрение схемы базы данных, привилегий. Управление пользователями баз данных. Рассмотрение вопросов, связанных с табличными пространствами и файлами данных. Модели и типы данных.
3. Рассмотрение вопросов, связанных со схемой и объектами схемы данных. Блоки данных, экстенты сегменты.
4. Рассмотрение вопросов, связанных с структурой памяти. Однопроцессорные и многопроцессорные базы данных.
5. Рассмотрение вопросов, связанных с транзакциями, блокировкой и согласованностью данных.
6. Журнал базы данных: структура и назначение файлов журнала, управление переключениями и контрольными точками.
7. Словарь данных: назначение, структура, префиксы.
8. Рассмотрение распределённых баз данных.
9. Выбор среды для реализации создаваемой учебной базы данных.
10. Определение обязанностей администратора применительно к создаваемой учебной базе данных.
11. Построение схемы создаваемой учебной базы данных.
12. Построение словаря создаваемой учебной базы данных.
13. Изучение команд администрирования данных, применительно к создаваемой учебной базе данных.
14. Разработка сценариев работы с данными в создаваемой учебной базе данных.
15. Рассмотрение вопросов, связанных с классификацией серверов, принципами разделения между клиентскими и серверными частями. Типовое разделение функций.

16. Рассмотрение вопросов, связанных с протоколами удаленного вызова процедур. Требования к аппаратным возможностям и базовому программному обеспечению клиентов и серверов.
17. Рассмотрение вопросов, связанных с хранимыми процедурами и триггерами.
18. Рассмотрение вопросов, связанных с характеристиками серверов баз данных и механизмами доступа к базам данных.
19. Рассмотрение вопросов, связанных с аппаратным обеспечением и развертыванием серверов баз данных.
20. Рассмотрение состава и схемы банка данных
21. Разработка технических требований к серверу баз данных, применительно к разрабатываемому заданию.
22. Разработка требований к корпоративной сети, применительно к разрабатываемому заданию.
23. Решение вопросов, связанных с конфигурированием сети, применительно к разрабатываемому заданию.
24. Разработка хранимых процедур, применительно к создаваемой учебной базе данных.
25. Разработка триггеров, применительно к создаваемой учебной базе данных.
26. Рассмотрение вопросов, связанных с технологией установки и настройка сервера MySQL в операционной системе Windows. Клиентские настройки, протоколирование, безопасность.
27. Рассмотрение вопросов, связанных с технологией установки и настройка сервера MySQL в операционных системах Linux.
28. Рассмотрение вопросов, связанных с удаленным администрированием.
29. Рассмотрение вопросов, связанных с аудитом базы данных, аудиторским журналом. Установка опций, включение и отключение аудита. Очистка и уменьшение размеров журнала.
30. Рассмотрение вопросов, связанных с технологиями создания базы данных с применением языка SQL. Добавление, удаление данных и таблиц.
31. Рассмотрение вопросов, связанных с созданием запросов, процедур и триггеров. Создание запросов и процедур на изменение структуры базы данных.

Задание выдал «13» января 2025 г. / М.Н. Витвицкий/

подпись

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ

РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«КУЗБАССКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИ-
ТЕТ ИМЕНИ Т.Ф.ГОРБАЧЕВА»

Филиал КузГТУ в г.Белово

ДНЕВНИК УЧЕБНОЙ ПРАКТИКЕ

профессионального модуля ПМ.07 «Сoadминистрирование баз данных и серверов»

студент Ботев Григорий Евгеньевич

группы ИС-225.2

специальности «09.02.07 Информационные системы и программирование»

Период практики с «13» января 2025 г. по «05» июня 2025.

База практики филиал КузГТУ в г.Белово

М.П.

Руководитель практики от

филиала КузГТУ в г.Белово

/ М.Н. Витвицкий/

подпись

Закончил практику «05» июня 2025.

Дата	Содержание работы	Оценка	Подпись
13.01.26	Рассмотрение обязанностей администратора баз данных. Основные утилиты администратора баз данных. Режимы запуска и останова базы данных.		
13.01.26, 20.01.26	Рассмотрение схемы базы данных, привилегий. Управление пользователями баз данных. Рассмотрение вопросов, связанных с табличными пространствами и файлами данных. Модели и типы данных.		
20.01.26, 27.01.26	Рассмотрение вопросов, связанных со схемой и объектами схемы данных. Блоки данных, экстен-ты сегменты.		
27.01.26	Рассмотрение вопросов, связанных с структурой памяти. Однопроцессорные и многопроцессор-ные базы данных.		
03.02.26	Рассмотрение вопросов, связанных с транзакци-ями, блокировкой и согласованностью данных.		
03.02.26	Журнал базы данных: структура и назначение файлов журнала, управление переключениями и контрольными точками.		
10.02.26	Словарь данных: назначение, структура, префик-сы.		
10.02.25	Рассмотрение распределённых баз данных.		
17.02.26	Выбор среды для реализации создаваемой учеб-ной базы данных.		
17.02.26	Определение обязанностей администратора при-менительно к создаваемой учебной базы данных.		
24.02.26	Построение схемы создаваемой учебной базы данных.		
24.02.26	Построение словаря создаваемой учебной базы		

	данных.		
03.03.26	Изучение команд администрирования данных, применительно к создаваемой учебной базе данных.		
03.03.26	Разработка сценариев работы с данными в создаваемой учебной базе данных.		
10.03.26	Рассмотрение вопросов, связанных с классификацией серверов, принципами разделения между клиентскими и серверными частями. Типовое разделение функций.		
10.03.26, 17.03.26	Рассмотрение вопросов, связанных с протоколами удаленного вызова процедур. Требования к аппаратным возможностям и базовому программному обеспечению клиентов и серверов.		
17.03.26	Рассмотрение вопросов, связанных с хранимыми процедурами и триггерами.		
24.03.26	Рассмотрение вопросов, связанных с характеристиками серверов баз данных и механизмами доступа к базам данных.		
24.03.26	Рассмотрение вопросов, связанных с аппаратным обеспечением и развертыванием серверов баз данных.		
24.03.26, 31.03.26	Рассмотрение состава и схемы банка данных		
31.03.26	Разработка технических требований к серверу баз данных, применительно к разрабатываемому заданию.		
31.03.26, 07.04.26	Разработка требований к корпоративной сети, применительно к разрабатываемому заданию.		
07.04.26	Решение вопросов, связанных с конфигурированием сети, применительно к разрабатываемому заданию.		
15.04.26,	Разработка хранимых процедур, применительно к		

16.04.26	создаваемой учебной базе данных.		
22.04.26, 24.04.26, 28.04.26	Разработка триггеров, применительно к создаваемой учебной базе данных.		
28.04.26, 30.04.26	Рассмотрение вопросов, связанных с технологией установки и настройка сервера MySQL в операционной системе Windows. Клиентские настройки, протоколирование, безопасность.		
30.04.26	Рассмотрение вопросов, связанных с технологией установки и настройка сервера MySQL в операционных системах Linux.		
30.04.26	Рассмотрение вопросов, связанных с удаленным администрированием.		
04.05.26	Рассмотрение вопросов, связанных с аудитом базы данных, аудиторским журналом. Установка опций, включение и отключение аудита. Очистка и уменьшение размеров журнала.		
05.05.26	Рассмотрение вопросов, связанных с технологиями создания базы данных с применением языка SQL. Добавление, удаление данных и таблиц.		
05.05.26	Рассмотрение вопросов, связанных с созданием запросов, процедур и триггеров. Создание запросов и процедур на изменение структуры базы данных.		

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	14
1.РАССМОТРЕНИЕ ОБЯЗАННОСТЕЙ АДМИНИСТРАТОРА БАЗ ДАННЫХ. ОСНОВНЫЕ УТИЛИТЫ АДМИНИСТРАТОРА БАЗ ДАННЫХ. РЕЖИМЫ ЗАПУСКА И ОСТАНОВА БАЗЫ ДАННЫХ.....	15
2.РАССМОТРЕНИЕ СХЕМЫ БАЗЫ ДАННЫХ, ПРИВЕЛЕГИЙ. УПРАВЛЕНИЕ ПОЛЬЗОВАТЕЛЯМИ БАЗ ДАННЫХ.....	17
3.РАССМОТРЕНИЕ ВОПРОСОВ, СВЯЗАННЫХ С ТАБЛИЧНЫМИ ПРОСТРАНСТВАМИ И ФАЙЛАМИ ДАННЫХ. МОДЕЛИ И ТИПЫ ДАННЫХ.....	18
4. РАССМОТРЕНИЕ ВОПРОСОВ, СВЯЗАННЫХ СО СХЕМОЙ И ОБЪЕКТАМИ СХЕМЫ ДАННЫХ. БЛОКИ ДАННЫХ, ЭКСТЕНТЫ СЕГМЕНТЫ.....	19
5. РАССМОТРЕНИЕ ВОПРОСОВ, СВЯЗАННЫХ СО СТРУКТУРОЙ ПАМЯТИ.	20
6. ОДНОПРОЦЕССОРНЫЕ И МНОГОПРОЦЕССОРНЫЕ БАЗЫ ДАННЫХ.....	21
7. РАССМОТРЕНИЕ ВОПРОСОВ, СВЯЗАННЫХ С ТРАНЗАКЦИЯМИ, БЛОКИРОВКОЙ И СОГЛАСОВАННОСТЬЮ ДАННЫХ.....	22
8. ЖУРНАЛ БАЗЫ ДАННЫХ: СТРУКТУРА И НАЗНАЧЕНИЕ ФАЙЛОВ ЖУРНАЛА, УПРАВЛЕНИЕ ПЕРЕКЛЮЧЕНИЯМИ И КОНТРОЛЬНЫМИ ТОЧКАМИ.....	23
9. СЛОВАРЬ ДАННЫХ: НАЗНАЧЕНИЕ, СТРУКТУРА, ПРЕФИКСЫ.....	24
10. РАССМОТРЕНИЕ РАСПРЕДЕЛЕННЫХ БАЗ ДАННЫХ.....	25
11. ВЫБОР СРЕДЫ ДЛЯ РЕАЛИЗАЦИИ РАЗРАБОТКИ.....	26
11.1 ВЫБОР СРЕДЫ ДЛЯ РЕАЛИЗАЦИИ СОЗДАВАЕМОЙ УЧЕБНОЙ БАЗЫ ДАННЫХ.....	26
11.2 ОПИСАНИЕ СРЕДЫ РАЗРАБОТКИ.....	28
12. ОПРЕДЕЛЕНИЕ ОБЯЗАННОСТЕЙ АДМИНИСТРАТОРА.....	29
12.1 ОПРЕДЕЛЕНИЕ ОБЯЗАННОСТЕЙ АДМИНИСТРАТОРА ПРИМЕНИТЕЛЬНО К СОЗДАВАЕМОЙ УЧЕБНОЙ БАЗЫ ДАННЫХ.....	29

					УП.2023.09.02.07.ПЗ			
Изм.	Лист	№ докум.	Подпись	Дата				
Разраб.		К.А.Димитринев			ПМ.07.01 Соадминистрирование баз данных и серверов СОДЕРЖАНИЕ	Лит.	Лист	Листов
Руковод.							9	90
от предп.		М.Н.Витвиц-				гр. ИС-225.2		

12.2 РУКОВОДСТВО АДМИНИСТРАТОРА БАЗЫ ДАННЫХ	30	13. ПОСТРОЕНИЕ СХЕМЫ БАЗЫ ДАННЫХ.....	32
13.1 ПОСТРОЕНИЕ СХЕМЫ СОЗДАВАЕМОЙ УЧЕБНОЙ БАЗЫ ДАННЫХ.....	32	13.2 ОПИСАНИЕ ER-МОДЕЛИ.....	33
14. ПОСТРОЕНИЕ СЛОВАРЯ БАЗЫ ДАННЫХ.....	34	14.1 ПОСТРОЕНИЕ СЛОВАРЯ СОЗДАВАЕМОЙ УЧЕБНОЙ БАЗЫ ДАННЫХ...	34
14.2 СЛОВАРЬ БАЗЫ ДАННЫХ.....	35	15. ИЗУЧЕНИЕ КОМАНД АДМИНИСТРИРОВАНИЯ ДАННЫХ.....	38
15.1 ИЗУЧЕНИЕ КОМАНД АДМИНИСТРИРОВАНИЯ ДАННЫХ, ПРИМЕНИТЕЛЬНО К СОЗДАВАЕМОЙ УЧЕБНОЙ БАЗЕ ДАННЫХ.....	38	15.2 ОПИСАНИЕ ОСНОВНЫХ КОМАНД ДЛЯ АДМИНИСТРИРОВАНИЯ ВАШЕЙ БАЗЫ ДАННЫХ С ИСПОЛЬЗОВАНИЕМ ВАШЕГО ИНТЕРФЕЙСА.....	39
16. РАЗРАБОТКА СЦЕНАРИЕВ РАБОТЫ С ДАННЫМИ.....	42	16.1 РАЗРАБОТКА СЦЕНАРИЕВ РАБОТЫ С ДАННЫМИ В СОЗДАВАЕМОЙ УЧЕБНОЙ БАЗЕ ДАННЫХ.....	42
16.2 СЦЕНАРИЙ ИСПОЛЬЗОВАНИЯ.....	43	17. РАССМОТРЕНИЕ ВОПРОСОВ, СВЯЗАННЫХ С КЛАССИФИКАЦИЕЙ СЕРВЕРОВ, ПРИНЦИПАМИ РАЗДЕЛЕНИЯ МЕЖДУ КЛИЕНТСКИМИ И СЕРВЕРНЫМИ ЧАСТЯМИ.....	45
18. ТИПОВОЕ РАЗДЕЛЕНИЕ ФУНКЦИЙ.....	46	19. РАССМОТРЕНИЕ ВОПРОСОВ, СВЯЗАННЫХ С ПРОТОКОЛАМИ УДАЛЕННОГО ВЫЗОВА ПРОЦЕДУР. ТРЕБОВАНИЯ К АППАРАТНЫМ ВОЗМОЖНОСТЯМ И БАЗОВОМУ ПРОГРАММНОМУ ОБЕСПЕЧЕНИЮ КЛИЕНТОВ И СЕРВЕРОВ.....	47
20. РАССМОТРЕНИЕ ВОПРОСОВ, СВЯЗАННЫХ С ХРАНИМЫМИ ПРОЦЕДУРАМИ И ТРИГГЕРАМИ.....	48	21. РАССМОТРЕНИЕ ВОПРОСОВ, СВЯЗАННЫХ С ХАРАКТЕРИСТИКАМИ СЕРВЕРОВ БАЗ ДАННЫХ И МЕХАНИЗМАМИ ДОСТУПА К БАЗАМ ДАННЫХ.....	49
22. РАССМОТРЕНИЕ ВОПРОСОВ, СВЯЗАННЫХ С АППАРАТНЫМ ОБЕСПЕЧЕНИЕМ И РАЗВЕРТЫВАНИЕМ СЕРВЕРОВ БАЗ ДАННЫХ.....	50	23. РАССМОТРЕНИЕ СОСТАВА И СХЕМЫ БАНКА ДАННЫХ.....	52

24.РАЗРАБОТКА ТЕХНИЧЕСКИХ ТРЕБОВАНИЙ К СЕРВЕРУ БАЗ ДАННЫХ	
24.2ТЕХНИЧЕСКИЕ ТРЕБОВАНИЯ К ПРОГРАММНЫМ СРЕДСТВАМ.....	55
25.РАЗРАБОТКА ТРЕБОВАНИЙ К КОРПОРАТИВНОЙ СЕТИ.....	56
25.1РАЗРАБОТКА ТРЕБОВАНИЙ К КОРПОРАТИВНОЙ СЕТИ, ПРИМЕНИТЕЛЬ- НО К РАЗРАБАТЫВАЕМОМУ ЗАДАНИЮ	56
25.2ТРЕБОВАНИЯ К ЛОКАЛЬНОЙ СЕТИ ДЛЯ ФУНКЦИОНИРОВАНИЯ ВАШЕ- ГО ИНТЕРФЕЙСА И БАЗЫ ДАННЫХ.....	57
26 РЕШЕНИЕ ВОПРОСОВ, СВЯЗАННЫХ С КОНФИГУРИРОВАНИЕМ СЕТИ..	59
26.1 РЕШЕНИЕ ВОПРОСОВ, СВЯЗАННЫХ С КОНФИГУРИРОВАНИЕМ СЕТИ, ПРИМЕНИТЕЛЬНО К РАЗРАБАТЫВАЕМОМУ ЗАДАНИЮ.....	59
26.2 КОНФИГУРАЦИЯ СЕТЕВОГО ОКРУЖЕНИЯ.....	60
27.РАЗРАБОТКА ХРАНИМЫХ ПРОЦЕДУР	61
28.РАЗРАБОТКА ТРИГГЕРОВ БАЗЫ ДАННЫХ.....	64
29.РАССМОТРЕНИЕ ВОПРОСОВ, СВЯЗАННЫХ С ТЕХНОЛОГИЕЙ УСТАНОВ- КИ И НАСТРОЙКА СЕРВЕРА MYSQL В ОПЕРАЦИОННОЙ СИСТЕМЕ WINDOWS. КЛИЕНТСКИЕ НАСТОЙКИ, ПРОТОКОЛИРОВАНИЕ, БЕЗОПАС- НОСТЬ.....	66
30.РАССМОТРЕНИЕ ВОПРОСОВ, СВЯЗАННЫХ С ТЕХНОЛОГИЕЙ УСТАНОВ- КИ СЕРВЕРА.....	67
31.1 РАССМОТРЕНИЕ ВОПРОСОВ, СВЯЗАННЫХ С УДАЛЕННЫМ АДМИНИ- СТРИРОВАНИЕМ.....	69
31.2ПРОЦЕСС УДАЛЕННОЙ РАБОТЫ.....	70
32.РАССМОТРЕНИЕ ВОПРОСОВ, СВЯЗАННЫХ С АУДИТОМ БАЗЫ ДАННЫХ. 72	
32.1 РАССМОТРЕНИЕ ВОПРОСОВ, СВЯЗАННЫХ С АУДИТОМ БАЗЫ ДАННЫХ, АУДИТОРСКИМ ЖУРНАЛОМ. УСТАНОВКА ОПЦИЙ, ВКЛЮЧЕНИЕ И ОТКЛЮ- ЧЕНИЕ АУДИТА. ОЧИСТКА И УМЕНЬШЕНИЕ РАЗМЕРОВ ЖУРНАЛА..... 72	
32.2СОЗДАНИЕ В ИНТЕРФЕЙСЕ КНОПКИ, ПОЗВОЛЯЮЩЕЙ ПРОИЗВЕСТИ АУДИТ И ЛОГИРОВАНИЕ ДЕЙСТВИЙ	73
33.РАССМОТРЕНИЕ ВОПРОСОВ, СВЯЗАННЫХ С ТЕХНОЛОГИЯМИ СОЗДА-	

НИЯ БАЗЫ ДАННЫХ	75
33.ОПИСАНИЕ КОМАНД ДОБАВЛЕНИЯ, УДАЛЕНИЯ, ИЗМЕНЕНИЯ ДАННЫХ И ТАБЛИЦ	77
34.РАССМОТРЕНИЕ ВОПРОСОВ, СВЯЗАННЫХ С СОЗДАНИЕ ЗАПРОСОВ, ПРОЦЕДУР И ТРИГГЕРОВ. СОЗДАНИЕ ЗАПРОСОВ И ПРОЦЕДУР НА ИЗМЕНЕНИЕ СТРУКТУРЫ БАЗЫ ДАННЫХ	78
35.РАССМОТРЕНИЕ ВОПРОСОВ, СВЯЗАННЫХ С ДИНАМИЧЕСКИМ SQL И ЕГО ОПЕРАТОРАМИ	79
ЗАКЛЮЧЕНИЕ	80
СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ	82

ВВЕДЕНИЕ

Практика имеет целью комплексное освоение обучающимися всех видов профессиональной деятельности по специальности (профессии) среднего профессионального образования, формирование общих и профессиональных компетенций, а также приобретение необходимых умений и опыта практической работы по специальности (профессии). Учебная практика по специальности направлена на формирование у обучающихся умений, приобретение первоначального практического опыта и реализуется в рамках профессиональных модулей ОП СПО по основным видам профессиональной деятельности для последующего освоения ими общих и профессиональных компетенций по избранной специальности.

					УП.2023.09.02.07.ПЗ	Лист
						13
Изм.	Лист	№ докум.	Подпись	Дата		

1. РАССМОТРЕНИЕ ОБЯЗАННОСТЕЙ АДМИНИСТРАТОРА БАЗ ДАННЫХ. ОСНОВНЫЕ УТИЛИТЫ АДМИНИСТРАТОРА БАЗ ДАННЫХ. РЕЖИМЫ ЗАПУСКА И ОСТАНОВА БАЗЫ ДАННЫХ

Администратор баз данных (DBA) отвечает за установку, настройку, поддержку и обеспечение безопасности баз данных. Основные обязанности:

1. Установка и настройка СУБД: Установка программного обеспечения и настройка параметров для оптимальной производительности.
2. Проектирование и создание баз данных: Разработка структуры базы данных (таблицы, индексы) и её оптимизация.
3. Управление пользователями и правами доступа: Создание учетных записей и назначение прав доступа.
4. Резервное копирование и восстановление: Организация резервного копирования и восстановление данных при сбоях.
5. Мониторинг и оптимизация: Анализ производительности и оптимизация запросов, индексов и конфигурации.
6. Обеспечение безопасности: Защита данных от несанкционированного доступа, шифрование и настройка аудита.

DBA использует специализированные утилиты для управления базами данных. Основные категории:

1. Управление базами данных:
 - SQLPlus (Oracle) — выполнение SQL-запросов.
 - pgAdmin (PostgreSQL) — графический инструмент.
 - MySQL Workbench (MySQL) — администрирование и разработка.
2. Резервное копировани
 - RMAN (Oracle) — резервное копирование и восстановление.
 - mysqldump (MySQL) — создание дампов.
3. Мониторинг и анализ:

- Oracle Enterprise Manager — мониторинг Oracle DB.
- SQL Server Management Studio (SSMS) — администрирование SQL Server.

4. Миграция данных:

- Oracle Data Pump — экспорт и импорт данных.
- AWS DMS — миграция данных в облако.

5. Обеспечение безопасности:

- Oracle Audit Vault — аудит и мониторинг.
- TDE (Transparent Data Encryption) — шифрование данных.

Режимы запуска и останова базы данных

Режимы запуска

1. Нормальный режим (OPEN): Полный доступ для чтения и записи.
2. Режим ограниченного доступа (RESTRICTED): Доступ только для администраторов.
3. Режим восстановления (RECOVER): Восстановление после сбоя.
4. Режим только для чтения (READ ONLY): Доступ только для чтения.
5. Режим монтирования (MOUNT): База данных подключена, но недоступна для пользователей.
6. Режим инициализации (NOMOUNT): Экземпляр СУБД запущен, база данных не подключена.

Режимы останова

1. Нормальный останов (NORMAL): Корректное завершение всех транзакций.
2. Транзакционный останов (TRANSACTIONAL): Завершение текущих транзакций, новые не допускаются.
3. Немедленный останов (IMMEDIATE): Прерывание активных транзакций.

2. РАССМОТРЕНИЕ СХЕМЫ БАЗЫ ДАННЫХ, ПРИВЕЛЕГИЙ. УПРАВЛЕНИЕ ПОЛЬЗОВАТЕЛЯМИ БАЗ ДАННЫХ

1. Схема базы данных и ее роль в безопасности

Схема базы данных представляет собой архитектурный план, определяющий структуру и организацию данных. Она включает в себя таблицы, столбцы, ключи и отношения между ними. Однако схема базы данных также играет важную роль в обеспечении безопасности.

В рамках схемы можно определить правила доступа и авторизации, которые контролируют, как пользователи взаимодействуют с данными. Например, в базе данных банка схема может включать таблицы для счетов, транзакций и клиентов. Администратор может установить ограничения, позволяющие сотрудникам службы поддержки видеть только информацию о счетах, но не о персональных данных клиентов.

2. Привилегии и разрешения

Привилегии являются фундаментальной концепцией в управлении доступом. Они представляют собой права, предоставляемые пользователям или группам для выполнения определенных действий с данными. Привилегии могут быть назначены на различные объекты базы данных, такие как таблицы, столбцы или даже отдельные строки.

Типичными привилегиями являются:

Чтение (SELECT): позволяет пользователям просматривать данные.

Вставка (INSERT): дает право добавлять новые записи.

Изменение (UPDATE): позволяет модифицировать существующие данные.

Удаление (DELETE): разрешает удалять записи.

Создание (CREATE): создание новых объектов базы данных.

3. РАССМОТРЕНИЕ ВОПРОСОВ, СВЯЗАННЫХ С ТАБЛИЧНЫМИ ПРОСТРАНСТВАМИ И ФАЙЛАМИ ДАННЫХ. МОДЕЛИ И ТИПЫ ДАННЫХ

В мире баз данных эффективное управление хранилищем и структурами данных является фундаментальным аспектом, влияющим на производительность и масштабируемость. В этой статье мы рассмотрим ключевые концепции, связанные с табличными пространствами, файлами данных и моделями баз данных, которые лежат в основе организации и хранения информации.

Табличные пространства и файлы данных

Табличное пространство (англ. Tablespace) представляет собой логическую структуру в базе данных, которая используется для управления размещением физических файлов данных. Оно обеспечивает гибкость в организации и хранении данных, позволяя администраторам базы данных настраивать структуру хранения в соответствии с требованиями приложения.

Табличные пространства содержат:

Файлы данных (Data Files): Физические файлы, используемые для хранения фактических данных таблиц и индексов. Файлы данных могут быть размещены на разных дисках или устройствах хранения, что позволяет распределять нагрузку и улучшать производительность.

Файлы журнала транзакций (Transaction Log Files): Файлы, в которых регистрируются изменения, вносимые в базу данных. Они играют важную роль в обеспечении целостности данных и возможности восстановления.

Администраторы могут создавать несколько табличных пространств для различных целей. Например, одно табличное пространство может быть предназначено для хранения пользовательских таблиц, в то время как другое - для индексов или временных данных. Это позволяет оптимизировать размещение данных и управление ими.

4. РАССМОТРЕНИЕ ВОПРОСОВ, СВЯЗАННЫХ СО СХЕМОЙ И ОБЪЕКТАМИ СХЕМЫ ДАННЫХ. БЛО- КИ ДАННЫХ, ЭКСТЕНТЫ СЕГМЕНТЫ

Объекты схемы базы данных

Схема базы данных состоит из нескольких ключевых объектов:

Таблицы (Tables): Основополагающие объекты, в которых хранятся данные. Таблица состоит из строк (записей) и столбцов (атрибутов).

Индексы (Indexes): Структуры, созданные для ускорения доступа к данным в таблицах. Индексы улучшают производительность запросов, обеспечивая быстрый поиск и сортировку данных.

Представления (Views): Виртуальные таблицы, основанные на результатах запросов. Представления могут агрегировать данные из нескольких таблиц и предоставлять специфические для приложения представления данных.

Хранимые процедуры и функции (Stored Procedures and Functions): Блоки кода, хранящиеся и выполняемые в базе данных. Они позволяют выполнять сложные операции и логику непосредственно в базе данных.

Синонимы (Synonyms): Альтернативные имена для объектов базы данных, упрощающие их обращение.

Экстенты и сегменты

Экстент - это группа смежных блоков данных, выделяемых вместе. Экстенты позволяют эффективно управлять пространством и обеспечивают непрерывное хранение данных. Размер экстента может варьироваться в зависимости от конфигурации базы данных.

Сегмент - это набор экстентов, выделенных для конкретного объекта базы данных, такого как таблица или индекс. Каждый объект имеет свой собственный сегмент, состоящий из одного или нескольких экстентов.

5. РАССМОТРЕНИЕ ВОПРОСОВ, СВЯЗАННЫХ СО СТРУКТУРОЙ ПАМЯТИ.

Уровни памяти

В системах баз данных можно выделить несколько уровней памяти, каждый из которых имеет свои характеристики и цели:

1. Кэш процессора (CPU Cache)

Кэш процессора - это высокоскоростная память, расположенная непосредственно на процессоре или рядом с ним. Он состоит из нескольких уровней (L1, L2, L3), каждый из которых имеет разный объем и скорость доступа. Кэш процессора хранит недавно использованные данные, обеспечивая их быструю доступность для процессора. Это критически важно для операций, связанных с обработкой запросов, таких как фильтрация, сортировка и агрегирование.

2. Оперативная память (RAM)

Оперативная память (RAM) является основным хранилищем, используемым для выполнения программ и хранения данных, с которыми они работают. Она обеспечивает быстрый доступ к данным, что значительно ускоряет обработку запросов. В контексте баз данных RAM используется для хранения буферов, кэшей и других структур данных, необходимых для выполнения запросов.

3. Дисковое хранилище (Disk Storage)

Дисковое хранилище представляет собой постоянное хранилище данных, которое используется для долгосрочного хранения информации. В базах данных дисковое хранилище используется для хранения самих данных, журналов транзакций и других файлов базы данных. Доступ к диску значительно медленнее, чем к RAM, поэтому оптимизация использования памяти и стратегии кэширования играют важную роль в повышении производительности.

6. ОДНОПРОЦЕССОРНЫЕ И МНОГОПРОЦЕССОРНЫЕ БАЗЫ ДАННЫХ

Однопроцессорные системы баз данных

Однопроцессорные системы баз данных, также известные как однопоточные системы, используют один центральный процессор (CPU) для обработки всех запросов и операций. Такие системы были преобладающими в ранних системах баз данных, когда вычислительные мощности были ограничены.

Характеристики однопроцессорных систем:

Последовательная обработка запросов: Запросы обрабатываются последовательно, один за другим. Каждый запрос ожидает завершения предыдущего, прежде чем начать выполнение.

Ограниченный параллелизм: Параллелизм в однопроцессорных системах ограничен многозадачностью операционной системы, которая позволяет выполнять несколько процессов одновременно, но все они используют один процессор.

Простота реализации: Однопроцессорные системы обычно имеют более простую архитектуру и реализацию, что упрощает управление и отладку.

Стратегии оптимизации:

Оптимизация запросов: Ключевым фактором в однопроцессорных системах является оптимизация каждого отдельного запроса. Это включает в себя использование индексов, оптимизацию плана выполнения запросов и сокращение количества ввода-вывода.

Управление блокировками: Эффективное управление блокировками и взаимной блокировкой (deadlocks) важно для предотвращения конфликтов при одновременном доступе к данным несколькими запросами.

7. РАССМОТРЕНИЕ ВОПРОСОВ, СВЯЗАННЫХ С ТРАНЗАКЦИЯМИ, БЛОКИРОВКОЙ И СОГЛАСОВАННОСТЬЮ ДАННЫХ.

Транзакции

Транзакция - это последовательность операций, выполняемых как единое целое. Она обеспечивает логическую целостность и атомарность, гарантируя, что либо все операции выполняются успешно, либо ни одна из них не будет применена. Транзакции являются основой для поддержания согласованности и надежности данных.

Характеристики транзакций:

Атомарность (Atomicity): Транзакция должна быть атомарной, то есть либо выполняется полностью, либо откатывается до первоначального состояния. Это предотвращает частичные изменения данных.

Согласованность (Consistency): Транзакция должна поддерживать согласованность данных, обеспечивая соблюдение всех ограничений и правил целостности.

Изолированность (Isolation): Транзакции должны быть изолированы друг от друга, чтобы изменения, сделанные одной транзакцией, не влияли на другие.

Длительность (Durability): После успешного завершения транзакции ее результаты должны быть сохранены и защищены от сбоев.

Управление транзакциями:

Начало и завершение: Каждая транзакция имеет четкое начало и конец, обозначаемые специальными командами или операциями.

Откат (Rollback): Если транзакция не может быть завершена успешно, она откатывается, возвращая данные к предыдущему согласованному состоянию

8. ЖУРНАЛ БАЗЫ ДАННЫХ: СТРУКТУРА И НАЗНАЧЕНИЕ ФАЙЛОВ ЖУРНАЛА, УПРАВЛЕНИЕ ПЕРЕКЛЮЧЕНИЯМИ И КОНТРОЛЬНЫМИ ТОЧКАМИ.

Структура журнала базы данных

Журнал базы данных представляет собой последовательность записей, которые документируют изменения, внесенные в базу данных. Каждая запись в журнале, часто называемая журнальной записью или журнальным блоком, содержит информацию о транзакции, изменившей данные. Структура журнала может варьироваться в зависимости от используемой СУБД, но обычно включает следующие элементы:

1. Идентификатор транзакции

Уникальный идентификатор, связывающий журнальную запись с конкретной транзакцией. Это позволяет отслеживать последовательность операций, выполняемых транзакцией.

2. Тип операции

Тип операции, такой как вставка (INSERT), обновление (UPDATE) или удаление (DELETE), указывает на то, какое действие было выполнено над данными.

3. Информация об измененных данных

Содержит информацию о том, какие данные были изменены. Это может включать в себя старые и новые значения, идентификаторы строк или другие метаданные, необходимые для восстановления состояния данных.

4. Временные метки

Временные метки используются для определения порядка операций и обеспечения согласованности. Они могут включать в себя время начала и окончания транзакции, а также временные отметки для каждой операции.

9. СЛОВАРЬ ДАННЫХ: НАЗНАЧЕНИЕ, СТРУКТУРА, ПРЕФИКСЫ.

Словарь данных (англ. Data Dictionary) является важной компонентой систем управления базами данных (СУБД), предоставляя централизованное хранилище метаданных о структуре базы данных. Он содержит подробную информацию о таблицах, столбцах, индексах, ограничениях и других объектах базы данных. В этой статье мы рассмотрим структуру словаря данных, его назначение и использование префиксов для идентификации объектов.

Назначение словаря данных

Словарь данных служит несколькими важными целями в системах баз данных:

Централизованное хранилище метаданных: Словарь данных содержит полную информацию о структуре базы данных, включая определения таблиц, столбцов, типов данных, ограничений и индексов. Он обеспечивает централизованное место для хранения и управления этой информацией.

Поддержка целостности: Словарь данных помогает обеспечивать целостность базы данных, храня информацию об ограничениях и правилах, которые должны соблюдаться. Он позволяет проверять и поддерживать целостность данных при внесении изменений в структуру базы данных.

Документация и аудит: Словарь данных служит документацией базы данных, предоставляя подробное описание ее структуры и объектов. Это облегчает понимание и обслуживание базы данных, а также аудит изменений.

Генерация кода: Словарь данных может использоваться для автоматической генерации кода, такого как SQL-скрипты для создания или изменения объектов базы данных. Это упрощает процесс разработки и развертывания.

10. РАССМОТРЕНИЕ РАСПРЕДЕЛЕННЫХ БАЗ ДАННЫХ

Распределенные базы данных представляют собой систему, в которой данные хранятся и обрабатываются на нескольких узлах или компьютерах, соединенных в сеть. В отличие от традиционных централизованных баз данных, где все данные находятся в одном месте, распределенные базы данных предлагают новые возможности для масштабируемости, доступности и эффективности. В этой статье мы рассмотрим структуру распределенных баз данных, их преимущества и ключевые концепции.

Структура распределенной базы данных

Распределенная база данных состоит из нескольких узлов, каждый из которых может быть отдельным компьютером или сервером. Эти узлы взаимодействуют друг с другом через сеть, обмениваясь данными и координируя операции. Структура распределенной базы данных может включать следующие компоненты:

1. Узлы (Nodes)

Узлы являются основными вычислительными единицами в распределенной базе данных. Каждый узел может иметь свою собственную локальную базу данных, а также ресурсы для обработки запросов. Узлы могут быть физически распределены на разных географических локациях или находиться в одном месте, но иметь отдельные вычислительные ресурсы.

2. Сетевое соединение (Network Connection)

Узлы в распределенной базе данных соединены друг с другом через сеть. Это может быть локальная сеть (LAN) или широкополосная сеть (WAN), в зависимости от требований к доступности и географическому охвату. Сетевое соединение позволяет узлам обмениваться данными, координировать операции и обеспечивать согласованность.

11. ВЫБОР СРЕДЫ ДЛЯ РЕАЛИЗАЦИИ РАЗРАБОТКИ

11.1 ВЫБОР СРЕДЫ ДЛЯ РЕАЛИЗАЦИИ СОЗДАВАЕМОЙ УЧЕБНОЙ БАЗЫ ДАННЫХ

При создании учебной базы данных важно выбрать подходящую среду, которая будет соответствовать целям обучения, обеспечивать необходимые функциональные возможности и предоставлять удобные инструменты для разработки и управления базой данных. В этой статье мы рассмотрим несколько вариантов сред для реализации учебной базы данных, их преимущества и факторы, которые следует учитывать при выборе.

Варианты сред для учебной базы данных

1. Локальная база данных (Local Database)

Локальная база данных устанавливается и запускается на локальном компьютере разработчика. Это может быть полноценная система управления базами данных (СУБД) или легковесный вариант, предназначенный для учебных целей. Примерами могут служить SQLite, MySQL для Windows, или встроенные базы данных, такие как H2 или SQLite Browser.

Преимущества:

Простота установки: Локальные базы данных обычно имеют простую установку и не требуют сложных конфигураций.

Быстрый доступ: Данные хранятся локально, что обеспечивает быстрый доступ и отсутствие задержек, связанных с сетью.

Самостоятельность: Разработчик имеет полный контроль над базой данных и может экспериментировать без влияния на другие системы.

2. Облачные базы данных (Cloud-based Databases)

Облачные базы данных предоставляются как сервис (DBaaS - Database as a Service) и размещаются на удаленных серверах провайдера. Пользователи могут создавать, управлять и использовать базы данных через веб-интерфейс или API. Примерами являются Amazon RDS, Google Cloud SQL, Microsoft Azure SQL Database.

Преимущества:

Легкость использования: Облачные базы данных не требуют установки и настройки, что упрощает начало работы.

Глобальная доступность: Данные доступны из любой точки мира через Интернет, что удобно для совместной работы и удаленного обучения.

Масштабируемость: Облачные провайдеры позволяют легко масштабировать ресурсы базы данных по мере необходимости.

Уровень сложности: Учитывайте уровень знаний и опыта обучающихся. Для начинающих подойдут простые и легковесные решения, в то время как для более продвинутых задач могут потребоваться полноценные СУБД.

Доступность данных: Рассмотрите, нужен ли доступ к данным из разных мест или достаточно локального доступа. Облачные решения обеспечивают глобальную доступность, в то время как локальные базы данных ограничены одним компьютером.

Ресурсы и стоимость: Некоторые среды могут требовать дополнительных ресурсов или иметь связанные с ними затраты. Облачные базы данных обычно предоставляются по подписке, в то время как локальные и виртуальные машины могут потребовать первоначальных затрат на программное обеспечение.

Инструменты разработки: Оцените, какие инструменты для разработки и администрирования предоставляет среда. Некоторые среды имеют удобные графические интерфейсы, в то время как другие требуют работы с командной строкой.

11.2 ОПИСАНИЕ СРЕДЫ РАЗРАБОТКИ

Visual Studio Code — это легковесный, но мощный редактор кода с открытым исходным кодом, разработанный Microsoft. Он поддерживает множество языков программирования, включая PHP, JavaScript, HTML, CSS и SQL.

Ключевые особенности VS Code:

Поддержка расширений

					УП.2023.09.02.07.ПЗ	Лист
						26
Изм.	Лист	№ докум.	Подпись	Дата		

- Доступны плагины для работы с PHP, MySQL, Git, Docker и другими технологиями.

- Например:

- PHP Intelephense (поддержка PHP)
- SQLTools (работа с базами данных)
- Live Server (запуск локального сервера для веб-разработки).

Встроенный терминал

- Позволяет выполнять команды без переключения между окнами.

Отладка

- Поддержка отладки PHP, JavaScript и других языков.

Интеграция с Git

- Встроенный контроль версий для работы с репозиториями.

Подсветка синтаксиса и IntelliSense

- Автодополнение кода, подсказки и проверка ошибок.

Работа с базами данных

- Через расширения (например, MySQL, PostgreSQL).

СУБД phpMyAdmin

phpMyAdmin — это веб-приложение для управления базами данных

MySQL/MariaDB. Оно позволяет:

- Создавать, изменять и удалять таблицы.
- Выполнять SQL-запросы.
- Импортировать и экспортировать данные (CSV, SQL, JSON).
- Управлять пользователями и правами доступа.

12. ОПРЕДЕЛЕНИЕ ОБЯЗАННОСТЕЙ АДМИНИСТРАТОРА

12.1 ОПРЕДЕЛЕНИЕ ОБЯЗАННОСТЕЙ АДМИНИСТРАТОРА ПРИМЕНИТЕЛЬНО К СОЗДАВАЕМОЙ УЧЕБНОЙ БАЗЫ ДАННЫХ

Администратор базы данных играет важную роль в управлении, обслуживании и обеспечении бесперебойной работы базы данных. В контексте учебной базы данных администратор отвечает за создание и поддержание среды, которая будет использоваться для обучения и демонстрации различных аспектов работы с базами данных. Ниже приведены ключевые обязанности администратора учебной базы данных.

1. Планирование и проектирование базы данных

Администратор должен спланировать и спроектировать структуру учебной базы данных, определив таблицы, столбцы, отношения и ограничения. Это включает в себя понимание требований к данным, которые будут храниться, и обеспечение соответствия структуры базы данных учебным целям. Администратор должен создать логическую схему базы данных, определяющую ее структуру, и физическую схему, описывающую ее реализацию.

2. Создание и настройка базы данных

После завершения проектирования администратор отвечает за создание базы данных. Это включает в себя выбор и настройку системы управления базами данных (СУБД), соответствующей учебным целям. Администратор должен установить и настроить СУБД, создать базу данных, таблицы и необходимые объекты, такие как индексы и ограничения.

12.2 РУКОВОДСТВО АДМИНИСТРАТОРА БАЗЫ ДАННЫХ

Настоящее техническое задание (ТЗ) определяет требования к структуре, содержанию и оформлению руководства администратора информационной системы (ИС). Руководство предназначено для обеспечения эффективного управления, настройки, мониторинга и поддержки информационной системы администраторами.

					УП.2023.09.02.07.ПЗ	Лист
						28
Изм.	Лист	№ докум.	Подпись	Дата		

Общие требования

2.1. Цель документа

Руководство администратора ИС должно предоставлять исчерпывающую информацию по установке, настройке, эксплуатации, обслуживанию и устранению неисправностей системы.

2.2. Целевая аудитория

Администраторы информационной системы, специалисты по технической поддержке, системные инженеры.

2.3. Язык документа

Руководство должно быть написано на русском языке. Терминология должна соответствовать общепринятым стандартам в области информационных технологий.

2.4. Формат документа

Документ должен быть предоставлен в электронном формате (PDF, DOCX) с возможностью печати. Допускается использование гиперссылок, таблиц, иллюстраций и диаграмм.

3. Структура документа

Руководство администратора ИС должно включать следующие разделы:

3.1. Титульный лист

- Название документа: "Руководство администратора информационной системы [название ИС]".

- Версия документа.

- Дата выпуска.

- Логотип организации (при наличии).

3.2. Оглавление

- Автоматически сгенерированное оглавление с указанием страниц.

3.3. Введение

- Краткое описание информационной системы.

- Назначение руководства.

- Описание целевой аудитории.

					УП.09.02.07.12 ПЗ	Лист
						29
Изм.	Лист	№ докум.	Подпись	Дата		

3.4. Общие сведения о системе

- Описание архитектуры системы.
- Основные компоненты системы.
- Требования к аппаратному и программному обеспечению.
- Поддерживаемые операционные системы и платформы.

3.5. Установка и настройка системы

- Пошаговая инструкция по установке системы.
- Настройка параметров системы.
- Интеграция с другими системами (при необходимости).
- Настройка прав доступа и ролей пользователей.

3.6. Эксплуатация системы

- Запуск и остановка системы.
- Управление пользователями и ролями.
- Мониторинг работы системы.
- Резервное копирование и восстановление данных.

13. ПОСТРОЕНИЕ СХЕМЫ БАЗЫ ДАННЫХ

13.1 ПОСТРОЕНИЕ СХЕМЫ СОЗДАВАЕМОЙ УЧЕБНОЙ БАЗЫ ДАННЫХ

Таблицы и их структура:

Таблица «Десерты» (Desserts)

DessertID: Уникальный идентификатор десерта (первичный ключ).

Название: Наименование десерта.

Описание: Состав и особенности десерта.

Цена: Стоимость единицы товара.

Количество_на_складе: Доступное для продажи количество.

CategoryID: Внешний ключ, ссылающийся на таблицу «Категории» (Categories).

Таблица «Категории» (Categories)

CategoryID: Уникальный идентификатор категории (первичный ключ).

Название_категории: Название группы десертов (например, «Торты», «Пирожные», «Печенье»).

Таблица «Поставщики» (Suppliers)

SupplierID: Уникальный идентификатор поставщика (первичный ключ).

Название_поставщика: Наименование компании или имя частного кондитера.

Контактное_лицо: ФИО представителя.

Телефон: Контактный номер телефона.

Таблица «Пользователи» (Users)

UserID: Уникальный идентификатор пользователя (первичный ключ).

Имя_пользователя: Логин для входа в систему.

Пароль: Зашифрованный пароль пользователя.

Роль: Роль в системе (например, «Клиент», «Кондитер», «Администратор»).

13.2 ОПИСАНИЕ ER-МОДЕЛИ

1. CUSTOMER (Клиенты)

Хранит информацию о покупателях: имя, фамилия, телефон, адрес, email.

Связан с заказами (ORDERS) по полю customer_id.

2. EMPLOYEES (Сотрудники)

Содержит данные о персонале: имя, фамилия, телефон, должность.

Связан через таблицу EMPLOYEE_ORDERS с заказами (многие ко многим), что позволяет закрепить за заказом, например, кондитера и курьера.

3. ORDERS (Заказы)

Основная таблица заказов.

Атрибуты: дата заказа, итоговая стоимость, статус выполнения, дата готовности и дата доставки/получения.

Связи:

С клиентами (один клиент — много заказов).

С сотрудниками через EMPLOYEE_ORDERS.

С десертами через ORDER_ITEMS.

С промокодами через PROMO_ORDER.

4. DESSERTS (Десерты)

Информация о товарах: название, описание, цена, количество на складе.

Связан с категориями (CATEGORIES) по полю category_id (один ко многим).

5. ORDER_ITEMS (Состав заказа)

Связывает десерты и заказы (многие ко многим).

Указывает количество каждого десерта в заказе и цену на момент оформления.

					УП.2023.09.02.07.ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		32

6. PROMOCODES (Промокоды)

Хранит информацию о скидочных купонах: код, размер скидки, срок действия.

7. PROMO_ORDER (Применённые промокоды)

Связывает промокоды и заказы (многие ко многим).

8. EMPLOYEE_ORDERS (Исполнители заказа)

Промежуточная таблица между EMPLOYEES и ORDERS.

Хранит роль сотрудника в конкретном заказе (например, «приготовил», «доставил»).

					УП.2023.09.02.07.ПЗ	Лист
						33
Изм.	Лист	№ докум.	Подпись	Дата		

14. ПОСТРОЕНИЕ СЛОВАРЯ БАЗЫ ДАННЫХ

14.1 ПОСТРОЕНИЕ СЛОВАРЯ СОЗДАВАЕМОЙ УЧЕБНОЙ БАЗЫ ДАННЫХ

Словарь данных является важным инструментом для хранения и управления метаданными о структуре базы данных. Ниже представлен словарь для учебной базы данных, описанной в предыдущем разделе. Этот словарь содержит подробную информацию о таблицах, столбцах, типах данных и ограничениях, что позволяет легко понимать и обслуживать базу данных.

Таблица 1.

Таблица	Поле	Тип данных	Описание	Ключ
Desserts	DessertID	Integer	Уникальный идентификатор десерта	PK
	Название	Varchar(255)	Название десерта	
	Описание	Text	Подробный состав и характеристики	
	Цена	Decimal(10, 2)	Цена за единицу товара	
	Количество_на_складе	Integer	Текущее количество товара на складе	
	CategoryID	Integer	Внешний ключ к таблице Categories	FK
Categories	CategoryID	Integer	Уникальный идентификатор категории	PK
	Название_категории	Varchar(100)	Название категории (например, «Пирожные», «Торты»)	
Suppliers	SupplierID	Integer	Уникальный идентификатор поставщика	PK
	Название_поставщика	Varchar(255)	Организация или ФИО кондитера	

Таблица	Поле	Тип данных	Описание	Ключ
	Контактное_лицо	Varchar(255)	Имя контактного лица	
	Телефон	Varchar(20)	Контактный телефон	
Users	UserID	Integer	Уникальный идентификатор пользователя	PK
	Имя_пользователя	Varchar(50)	Логин для входа	
	Пароль	Varchar(255)	Хэш пароля	
	Роль	Varchar(20)	Роль (Клиент, Кондитер, Администратор)	

14.2 СЛОВАРЬ БАЗЫ ДАННЫХ

Таблица 2. словарь базы данных

Таблица	Поле	Тип данных	Описание	Ключ
Customers	customer_id	INT	Уникальный идентификатор клиента	PK
	first_name	VARCHAR(50)	Имя клиента	
	last_name	VARCHAR(50)	Фамилия клиента	
	phone	VARCHAR(20)	Номер телефона	

Таблица	Поле	Тип данных	Описание	Ключ
	email	VARCHAR(100)	Электронная почта	
	address	VARCHAR(255)	Адрес доставки	
Orders	order_id	INT	Уникальный идентификатор заказа	PK
	customer_id	INT	Клиент, оформивший заказ	FK → Customers
	order_date	DATETIME	Дата и время оформления	
	total_cost	DECIMAL(10,2)	Итоговая стоимость заказа	
	status	VARCHAR(50)	Статус (новый, готовится, доставлен и др.)	
	completion_date	DATE	Плановая или фактическая дата готовности	
	delivery_date	DATE	Дата доставки / получения	
Desserts	dessert_id	INT	Уникальный идентификатор десерта	PK
	name	VARCHAR(255)	Название десерта	
	description	TEXT	Описание и состав	
	price	DECIMAL(10,2)	Цена	

					УП.09.02.07.14 ПЗ	Лист
						36
Изм.	Лист	№ докум.	Подпись	Дата		

Таблица	Поле	Тип данных	Описание	Ключ
		0,2)		
	stock_quantity	INT	Остаток на складе	
	category_id	INT	Категория десерта	FK → Categories
Categories	category_id	INT	Уникальный идентификатор категории	PK
	name	VARCHAR(100)	Название категории (например, «Торты», «Капкейки»)	
Order_Items	order_item_id	INT	Уникальный идентификатор позиции заказа	PK
	order_id	INT	Заказ	FK → Orders
	dessert_id	INT	Десерт	FK → Desserts
	quantity	INT	Количество	
	price_at_order	DECIMAL(10,2)	Цена десерта на момент заказа (фиксация)	
Employees	employee_id	INT	Уникальный идентификатор сотрудника	PK
	first_name	VARCHAR(50)	Имя сотрудника	
	last_name	VARCHAR(50)	Фамилия сотрудника	

Таблица	Поле	Тип данных	Описание	Ключ
	phone	VARCHAR(20)	Телефон	
	position	VARCHAR(100)	Должность (кондитер, курьер, менеджер)	
Employee_Orders	employee_order_id	INT	Уникальный идентификатор записи	PK
	employee_id	INT	Сотрудник	FK → Employees
	order_id	INT	Заказ	FK → Orders
	role	VARCHAR(50)	Роль в выполнении заказа («приготовил», «доставил»)	
Promocodes	promo_id	INT	Уникальный идентификатор промокода	PK
	code	VARCHAR(50)	Символьный код купона	
	discount	DECIMAL(5,2)	Скидка (в процентах или фиксированная сумма)	
	valid_from	DATE	Начало действия	
	valid_until	DATE	Окончание действия	
Promo_Order	promo_order_id	INT	Уникальный идентификатор связи	PK

Таблица	Поле	Тип данных	Описание	Ключ
	order_id	INT	Заказ	FK → Orders
	promo_id	INT	Применённый промокод	FK → Promocodes

OPEN EDITORS

index.php Project_Dimitrinev(logs)

PROJECT_DIMITRINEV(...)

Project_Dimitrinev(logs)

css

images

logs

sounds

advertising_points_in_order.php

advertising_points.php

clear_logs.php

clients.php

dashboard.php

db_connect.php

export_xls.php

get_logs.php

index.php

launch-remmina.php

log_action_endpoint.php

logger.php

logout.php

materials_in_order.php

materials.php

orders.php

process_sql.php

project_Dimitrinev.sql

staff_orders.php

staff.php

test_and_export_users.php

Project_Dimitrinev(logs) > index.php

```
1 <?php
2 session_start();
3
4 // Подключаем файл с настройками подключения к базе данных
5 include 'db_connect.php';
6
7 if ($_SERVER['REQUEST_METHOD'] == 'POST') {
8     // Получаем логин и пароль из формы
9     $login = $_POST['phone'];
10    $password = $_POST['password'];
11
12    // Подготовленный запрос для поиска пользователя в базе данных
13    $stmt = $pdo->prepare("SELECT * FROM users WHERE login = :phone AND password = :password");
14    $stmt->bindParam(':phone', $login);
15    $stmt->bindParam(':password', $password);
16    $stmt->execute();
17
18    // Проверяем, существует ли пользователь с такими данными
19    if ($stmt->rowCount() > 0) {
20        $_SESSION['user'] = $login; // Записываем логин в сессию
21        header("Location: dashboard.php"); // Перенаправляем на главную панель
22        exit();
23    } else {
24        $error = "Неверный логин или пароль";
25    }
26 }
27 ?>
28
29 <!DOCTYPE html>
30 <html lang="ru">
31 <head>
32     <meta charset="UTF-8">
33     <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

Рисунок 1.

15. ИЗУЧЕНИЕ КОМАНД АДМИНИСТРИРОВАНИЯ ДАННЫХ

15.1 ИЗУЧЕНИЕ КОМАНД АДМИНИСТРИРОВАНИЯ ДАННЫХ, ПРИМЕНИТЕЛЬНО К СОЗДАВАЕМОЙ УЧЕБНОЙ БАЗЕ ДАННЫХ

Команды администрирования учебной базы данных

Администрирование базы данных включает в себя множество задач, от создания и настройки базы данных до управления пользователями, обеспечением безопасности и оптимизацией производительности. В этом разделе мы рассмотрим некоторые основные команды администрирования, применимые к учебной базе данных, описанной в предыдущих разделах.

1. Создание базы данных и таблиц

Для создания учебной базы данных и необходимых таблиц можно использовать следующие команды SQL:

-- Создание базы данных

```
CREATE DATABASE LibraryDB;
```

-- Переключение на созданную базу данных

```
USE LibraryDB;
```

-- Создание таблицы "Книги"

```
CREATE TABLE Books (  
    BookID INT PRIMARY KEY,  
    Название VARCHAR(255),  
    Автор VARCHAR(255),  
    Год_издания INT,  
    Количество_экземпляров INT,  
    Доступно INT  
);
```

15.2 ОПИСАНИЕ ОСНОВНЫХ КОМАНД ДЛЯ АДМИНИСТРИРОВАНИЯ ВАШЕЙ БАЗЫ ДАННЫХ С ИСПОЛЬЗОВАНИЕМ ВАШЕГО ИНТЕРФЕЙСА

1. Управление таблицами (CRUD - Create, Read, Update, Delete):

- Кнопки на странице, ведущие на materials.php, clients.php, advertising_points.php, и т.д., позволяют администратору управлять данными в соответствующих таблицах базы данных.

- Предполагается, что страницы materials.php, clients.php и подобные содержат формы и скрипты для выполнения основных операций CRUD над таблицами:

- Create (Создание): Добавление новых записей в таблицу. Например, добавить новый материал, нового клиента или новую рекламную точку. Вероятно, на этих страницах есть формы для ввода данных и кнопки “Сохранить” или “Добавить”.

- Read (Чтение): Просмотр существующих записей в таблице. Обычно это реализуется в виде таблицы на странице, отображающей данные из базы.

- Update (Обновление): Редактирование существующих записей. Вероятно, каждая строка в таблице имеет кнопку “Редактировать”, которая открывает форму с заполненными данными, которые можно изменить и сохранить.

- Delete (Удаление): Удаление существующих записей. Как правило, возле каждой записи есть кнопка “Удалить”.

- Запросы к базе данных: Внутри materials.php, clients.php и подобных, PHP-скрипты выполняют SQL-запросы к базе данных для реализации CRUD-операций:

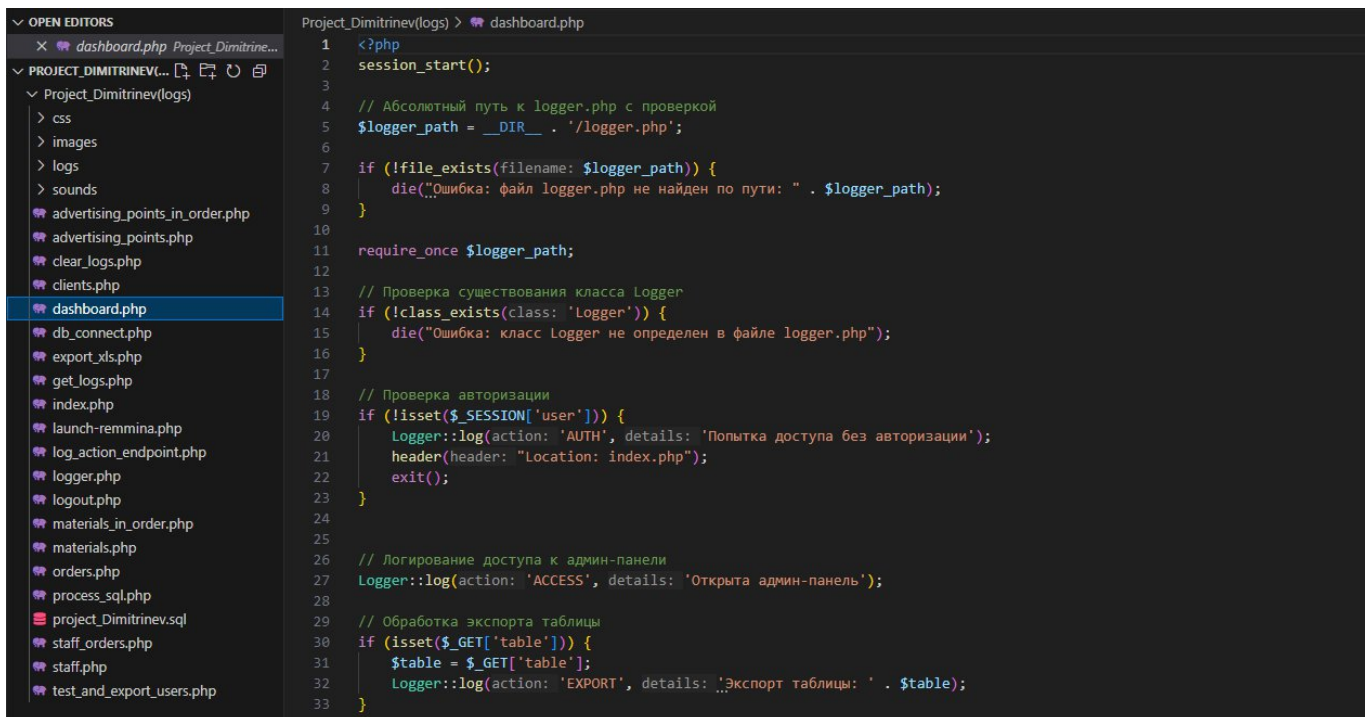


Рисунок 2. Список файлов

2. Консоль SQL:

- Возможные команды: Через консоль SQL можно выполнять любые команды SQL, которые поддерживает СУБД (например, MySQL): SELECT, INSERT, UPDATE, DELETE, CREATE TABLE, ALTER TABLE, DROP TABLE, и т.д. Это предоставляет гибкий способ администрирования базы данных, но также требует от пользователя знания SQL.

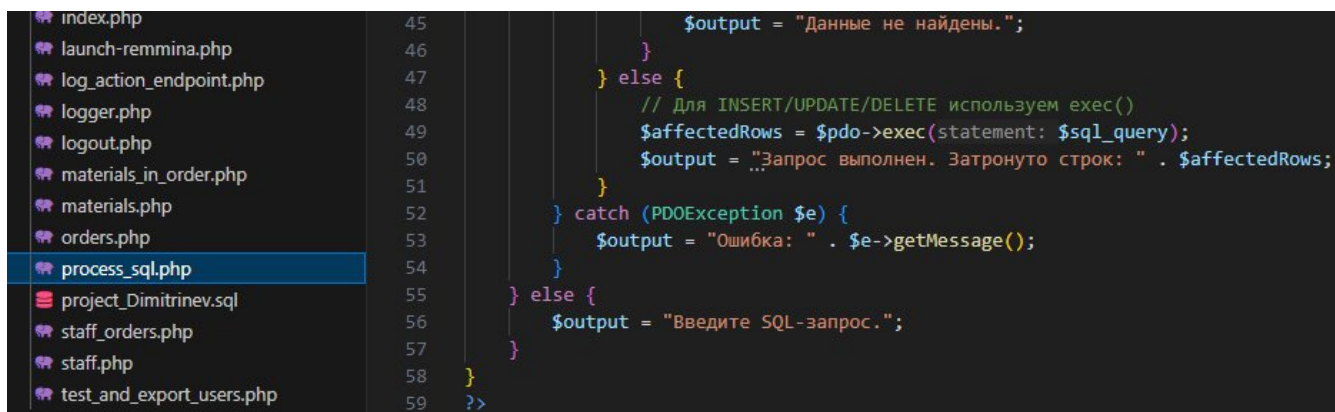


Рисунок 3. Код SQL

3. Экспорт в XLS (Excel):

- Основные команды:
- SELECT * FROM <table_name>: Скрипт export_xls.php сначала выполняет этот SQL-запрос, чтобы получить все данные из указанной таблицы.

- Затем, скрипт преобразует полученные данные в формат XLS и отправляет файл пользователю для скачивания.

```

> images      11
> logs        12
> sounds      13
advertising_points_in_order.php 14
advertising_points.php           15
clear_logs.php                   16
clients.php                      17
dashboard.php                   18
db_connect.php                  19
export_xls.php                   20
get_logs.php                    21
index.php                      22
launch-remmina.php             23
log_action_endpoint.php        24
logger.php                     25
logout.php                     26
materials_in_order.php         27
materials.php                  28
orders.php                     29
process_sql.php                30

```

```

11 $table_names = [
12     'Advertising_points' => 'Рекламные места',
13     'Advertising_points_in_order' => 'Рекламные места в заказе',
14     'Client' => 'Клиенты',
15     'Materials' => 'Материалы',
16     'Materials_in_the_order' => 'Материалы в заказе',
17     'Orders' => 'Заказы',
18     'Staff' => 'Сотрудники',
19     'Staff_Orders' => 'Сотрудники в заказах',
20     'users' => 'Пользователи'
21 ];
22
23
24
25 try {
26     $pdo = new PDO(dsn: "mysql:host=$host;dbname=$dbname", username: $username, password: $password);
27     $pdo->setAttribute(attribute: PDO::ATTR_ERRMODE, value: PDO::ERRMODE_EXCEPTION);
28
29     // Получение данных из таблицы
30     $stmt = $pdo->query("SELECT * FROM $tableName");
31     $data = $stmt->fetchAll(PDO::FETCH_ASSOC);
32
33     if (empty($data)) {
34         die("Таблица пуста или не существует.");
35     }
36 }

```

- Рисунок 4. Экспорт

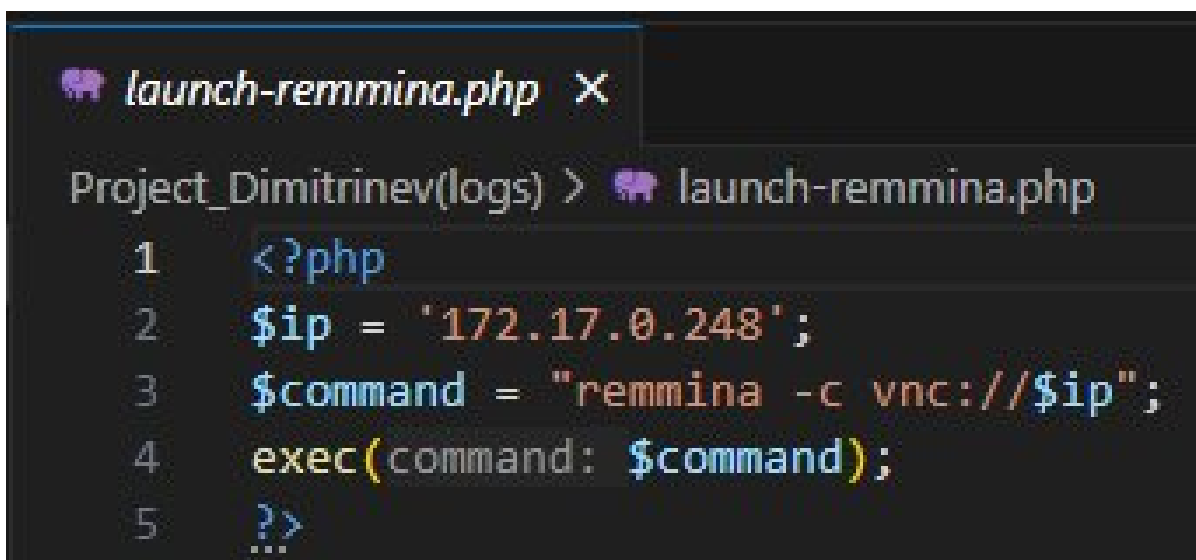
4. Запуск Remmina (VNC):

- Кнопка `<a href="launch-remmina.php" target="_blank" class="button vnc-button">Открыть VNC</a>` предназначена для открытия VNC-клиента (Remmina) для удаленного доступа к серверу.

- При нажатии на кнопку выполняется AJAX-запрос к `launch-remmina.php`.

- Предполагается, что `launch-remmina.php` пытается запустить Remmina на сервере (а не на компьютере пользователя). Это может быть сделано с помощью системных команд PHP (например, `exec()`). Важно: Запуск программ на сервере через веб-интерфейс может быть опасным и требует тщательной проверки безопасности.

- Скрипт выводит сообщение об успешном или неуспешном запуске Remmina.



```
launch-remmina.php X
Project_Dimitrinev(logs) > launch-remmina.php
1  <?php
2  $ip = '172.17.0.248';
3  $command = "remmina -c vnc://$ip";
4  exec(command: $command);
5  ?>
```

• Рисунок 5. Запуск Reminna

5. Выход:

- Форма `<form action="logout.php" method="post">` позволяет пользователю выйти из административной панели.

16. РАЗРАБОТКА СЦЕНАРИЕВ РАБОТЫ С ДАННЫМИ

16.1 РАЗРАБОТКА СЦЕНАРИЕВ РАБОТЫ С ДАННЫМИ В СОЗДАВАЕМОЙ УЧЕБНОЙ БАЗЕ ДАННЫХ

Учебная база данных предоставляет отличную возможность для изучения и практического применения различных операций с базами данных. Ниже представлены несколько сценариев работы с данными, которые могут быть реализованы в учебной базе данных, описанной в предыдущих разделах.

1. Добавление новых десертов

Сценарий: Менеджер магазина получил новую партию десертов и должен внести их в базу данных.

Действия:

Менеджер открывает интерфейс управления базой данных (например, SQL-клиент или веб-интерфейс).

Он выполняет запрос для добавления новых десертов, указывая их название, описание, цену и количество на складе.

Система проверяет данные на соответствие ограничениям (например, наличие указанной категории в таблице «Категории», корректность цены) и добавляет новые записи в таблицу «Десерты».

Менеджер может проверить, были ли десерты добавлены успешно, выполнив запрос на поиск по названию или категории.

16.2 СЦЕНАРИЙ ИСПОЛЬЗОВАНИЯ

Вход в административную панель:

Пользователь открывает страницу `index.php` в браузере. Эта страница, скорее всего, содержит форму для ввода логина и пароля.

Пользователь вводит свои учетные данные и нажимает кнопку “Войти”.

Скрипт проверяет учетные данные в таблице `Users` базы данных. Если учетные данные верны, создается сессия пользователя (`$_SESSION['user']`) и пользователь перенаправляется на страницу `dashboard.php`.

Если учетные данные неверны, отображается сообщение об ошибке, и пользователь остается на странице входа.

Просмотр информации на панели управления (`dashboard.php`):

После успешного входа пользователь попадает на страницу `dashboard.php`.

На странице отображается приветствие (например, “Добро пожаловать, [Имя пользователя]!”).

Отображается информационный блок с общим описанием системы, указанием разработчиков и защитой авторских прав.

Воспроизводится фоновая музыка (которую можно отключить кнопкой).

Отображаются кнопки для управления различными таблицами базы данных.

Управление данными в таблице «Десерты» (`desserts.php`):

На странице `dashboard.php` пользователь нажимает кнопку “Управление таблицей ‘Десерты’”. Открывается страница `desserts.php`.

На странице `desserts.php` отображается таблица со списком всех десертов из базы данных.

Пользователь может выполнить следующие действия:

Добавить новый десерт: Нажимает кнопку “Добавить десерт”, заполняет форму с данными о десерте (название, описание, цена, количество на складе, категория) и нажимает кнопку “Сохранить”. Данные добавляются в таблицу `Desserts`.

Редактировать существующий десерт: Находит десерт в таблице, нажимает кнопку “Редактировать” (или иконку редактирования), редактирует данные в форме и нажимает кнопку “Сохранить”. Данные десерта обновляются в таблице `Desserts`.

Удалить десерт: Находит десерт в таблице и нажимает кнопку “Удалить” (или иконку удаления). Десерт удаляется из таблицы `Desserts`.

Примечание: Аналогичные сценарии применимы для управления другими таблицами: `customers.php` (Клиенты), `categories.php` (Категории), `orders.php` (Заказы), `employees.php` (Сотрудники), `order_items.php` (Состав заказа), `employee_orders.php` (Исполнители заказа), `promocodes.php` (Промокоды).

Выполнение SQL-запроса через консоль SQL (`process_sql.php`):

					УП.09.02.07.16 ПЗ	Лист
						46
Изм.	Лист	№ докум.	Подпись	Дата		

На странице dashboard.php пользователь находит раздел “SQL Console”.

В текстовое поле пользователь вводит SQL-запрос (например, `SELECT * FROM Desserts WHERE Цена < 500`).

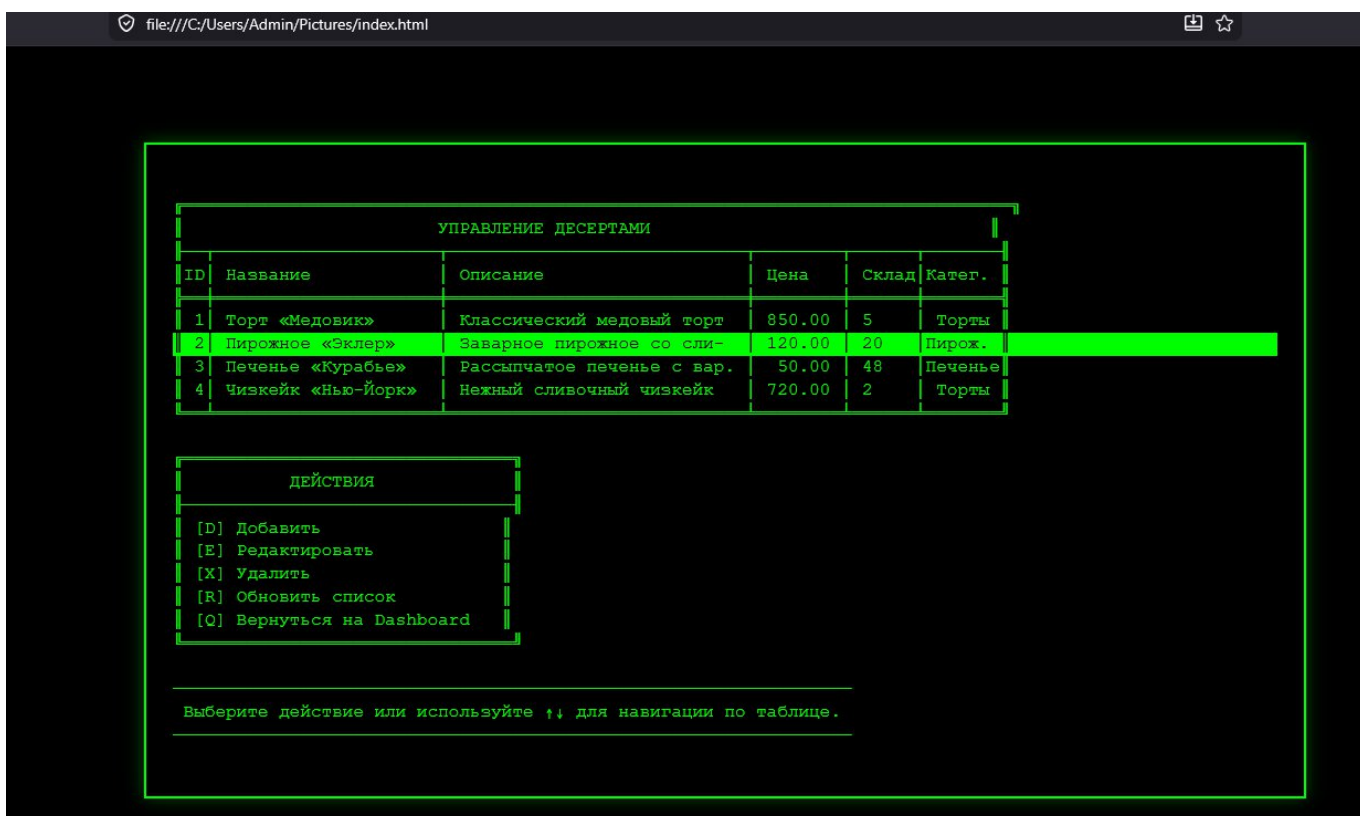
Пользователь нажимает кнопку “Выполнить запрос”.

Скрипт process_sql.php выполняет запрос на сервере базы данных.

Если запрос `SELECT`, результаты отображаются на странице в виде таблицы.

Если запрос `INSERT`, `UPDATE` или `DELETE`, отображается сообщение об успешном выполнении или об ошибке.

Пример: Пользователь вводит `SELECT COUNT(*) FROM Desserts`, чтобы узнать общее количество десертов в ассортименте.



УПРАВЛЕНИЕ ДЕСЕРТАМИ					
ID	Название	Описание	Цена	Склад	Катег.
1	Торт «Медовик»	Классический медовый торт	850.00	5	Торты
2	Пирожное «Эклер»	Заварное пирожное со сли-	120.00	20	Пирож.
3	Печенье «Курабье»	Рассыпчатое печенье с вар.	50.00	48	Печенье
4	Чизкейк «Нью-Йорк»	Нежный сливочный чизкейк	720.00	2	Торты

действия

[D] Добавить

[E] Редактировать

[X] Удалить

[R] Обновить список

[Q] Вернуться на Dashboard

Выберите действие или используйте ↑↓ для навигации по таблице.

Рисунок 6. Интерфейс

17. РАССМОТРЕНИЕ ВОПРОСОВ, СВЯЗАННЫХ С КЛАССИФИКАЦИЕЙ СЕРВЕРОВ, ПРИНЦИПАМИ РАЗДЕЛЕНИЯ МЕЖДУ КЛИЕНТСКИМИ И СЕРВЕРНЫМИ ЧАСТЯМИ

В современных системах баз данных часто используется архитектура клиент-сервер, которая разделяет функциональность между клиентской частью и сервером. В этой статье мы рассмотрим классификацию серверов баз данных и принципы разделения обязанностей между клиентом и сервером.

Классификация серверов баз данных

Серверы баз данных можно классифицировать по нескольким критериям, включая масштаб, функциональность и область применения. Ниже представлены некоторые распространенные типы серверов:

1. Серверы баз данных общего назначения (General-Purpose Database Servers)

Эти серверы предназначены для широкого спектра задач и могут использоваться в различных приложениях. Они предлагают полный набор функций управления базами данных, включая хранение данных, обработку запросов, транзакции и безопасность. Примерами являются MySQL, PostgreSQL, Microsoft SQL Server.

2. Серверы для аналитики и больших данных (Analytics and Big Data Servers)

Серверы этой категории специализируются на обработке больших объемов данных и аналитических задач. Они оптимизированы для работы с большими наборами данных, часто используют распределенные системы и поддерживают обработку данных в реальном времени. Примерами являются Apache Hadoop, Apache Spark, и спец

3. Серверы для управления содержимым (Content Management Servers)

Эти серверы предназначены для управления и доставки контента, такого как документы, медиа-файлы и веб-страницы. Они часто используются в системах управления контентом (CMS).

18. ТИПОВОЕ РАЗДЕЛЕНИЕ ФУНКЦИЙ

В архитектуре клиент-сервер функции и обязанности четко разделены между клиентской частью и сервером. Это разделение позволяет оптимизировать работу системы, обеспечивая эффективное использование ресурсов, безопасность и масштабируемость. Ниже представлено типичное разделение функций в такой архитектуре.

Функции клиента:

1. Интерфейс пользователя (User Interface)

Клиент отвечает за создание и управление интерфейсом пользователя, который может включать графический интерфейс (GUI) или веб-интерфейс. Интерфейс обеспечивает взаимодействие пользователя с системой, позволяя вводить запросы, просматривать результаты и управлять данными.

2. Ввод и отображение данных (Data Input and Display)

Клиент обрабатывает ввод данных пользователем, таких как поисковые запросы, фильтры или новые записи. Он также отвечает за отображение результатов запросов, таблиц и графиков. Клиент может использовать различные элементы интерфейса, такие как формы, таблицы, диаграммы, для представления данных.

3. Локальное кэширование (Local Caching)

Клиент может кэшировать некоторые данные локально, чтобы уменьшить количество запросов к серверу. Например, он может хранить недавно просмотренные записи или часто используемые данные, что позволяет ускорить доступ и снизить нагрузку на сервер.

4. Валидация данных (Data Validation)

Клиент может выполнять валидацию введенных данных, проверяя их на соответствие правилам и ограничениям. Это включает в себя проверку формата, диапазона значений и обязательных полей. Валидация на клиенте позволяет быстро выявлять ошибки и обеспечивать целостность данных.

19. РАССМОТРЕНИЕ ВОПРОСОВ, СВЯЗАННЫХ С ПРОТОКОЛАМИ УДАЛЕННОГО ВЫЗОВА ПРОЦЕДУР. ТРЕБОВАНИЯ К АППАРАТНЫМ ВОЗМОЖНОСТЯМ И БАЗОВОМУ ПРОГРАММНОМУ ОБЕСПЕЧЕНИЮ КЛИЕНТОВ И СЕРВЕРОВ

Протоколы удаленного вызова процедур (Remote Procedure Call, RPC) являются важной частью архитектуры клиент-сервер, позволяя клиентам вызывать функции на сервере, как если бы они были локальными. В этой статье мы рассмотрим протоколы RPC, их роль в архитектуре клиент-сервер и требования к аппаратному и программному обеспечению клиентов и серверов.

Протоколы удаленного вызова процедур (RPC)

RPC - это механизм, позволяющий одной программе или процессу вызывать функции или процедуры, расположенные в другом адресном пространстве, обычно на удаленном компьютере. RPC абстрагирует сетевое взаимодействие, позволяя клиентам вызывать функции на сервере так, как будто они были локальными.

Принцип работы RPC:

1. Клиент вызывает удаленную процедуру, передавая ей параметры.
2. RPC система на клиенте упаковывает параметры в сообщение и отправляет его серверу.
3. RPC система на сервере получает сообщение, распаковывает параметры и вызывает соответствующую процедуру.
4. Сервер выполняет процедуру и возвращает результат.
5. RPC система на сервере упаковывает результат в сообщение и отправляет его клиенту.
6. RPC система на клиенте получает сообщение, распаковывает результат и возвращает его вызывающей программе.

20. РАССМОТРЕНИЕ ВОПРОСОВ, СВЯЗАННЫХ С ХРАНИМЫМИ ПРОЦЕДУРАМИ И ТРИГГЕРАМИ

Хранимые процедуры и триггеры являются мощными инструментами в арсенале администратора и разработчика баз данных. Они позволяют автоматизировать задачи, улучшать производительность, обеспечивать целостность данных и расширять функциональность базы данных. В этой статье мы рассмотрим, что представляют собой хранимые процедуры и триггеры, их преимущества и примеры использования.

Хранимые процедуры (Stored Procedures)

Хранимые процедуры - это заранее определенные наборы SQL-команд, которые хранятся и выполняются на сервере базы данных. Они позволяют группировать логику, связанную с базой данных, и выполнять сложные операции с данными.

Преимущества хранимых процедур:

Повышение производительности: Хранимые процедуры могут быть оптимизированы для конкретной базы данных, используя индексы, кэширование и другие техники. Это может значительно улучшить производительность по сравнению с выполнением отдельных SQL-запросов.

Безопасность: Хранимые процедуры могут ограничивать прямой доступ к таблицам, обеспечивая более высокий уровень безопасности. Пользователи могут вызывать процедуры, не имея прав на изменение данных.

Повторное использование кода: Логика, реализованная в хранимых процедурах, может быть повторно использована в разных частях приложения, что упрощает обслуживание и развитие системы.

Сокращение сетевого трафика: Хранимые процедуры выполняются на сервере, что сокращает количество сетевых запросов по сравнению с выполнением отдельных SQL-команд.

**21.РАССМОТРЕНИЕ ВОПРОСОВ, СВЯЗАННЫХ С
ХАРАКТЕРИСТИКАМИ СЕРВЕРОВ БАЗ ДАННЫХ И МЕ-
ХАНИЗМАМИ ДОСТУПА К БАЗАМ ДАННЫХ**

Серверы баз данных являются критически важными компонентами в системах управления базами данных (СУБД), отвечающими за хранение, обработку и обеспечение доступа к данным. В этой статье мы рассмотрим основные характеристики серверов баз данных и механизмы доступа, которые позволяют клиентам взаимодействовать с базой данных.

Характеристики серверов баз данных

Серверы баз данных имеют несколько ключевых характеристик, которые влияют на их производительность, масштабируемость и надежность:

1. Скорость процессора (CPU Speed)

Скорость процессора влияет на скорость обработки запросов и операций с базой данных. Более быстрые процессоры позволяют серверу обрабатывать больше запросов в единицу времени.

2. Объем памяти (Memory Capacity)

Объем памяти, обычно измеряемый в гигабайтах (GB), влияет на количество данных, которые сервер может хранить в оперативной памяти. Больше памяти позволяет эффективнее кэшировать данные и улучшать производительность.

3. Емкость хранилища (Storage Capacity)

Емкость хранилища определяет, сколько данных сервер может хранить на диске. Это может включать в себя жесткие диски, твердотельные накопители (SSD) или сетевые хранилища. Серверы с большей емкостью могут хранить большие базы данных.

22. РАССМОТРЕНИЕ ВОПРОСОВ, СВЯЗАННЫХ С АППАРАТНЫМ ОБЕСПЕЧЕНИЕМ И РАЗВЕРТЫВАНИЕМ СЕРВЕРОВ БАЗ ДАННЫХ

Развертывание серверов баз данных требует тщательного планирования и понимания требований к аппаратному обеспечению для обеспечения оптимальной производительности и надежности. В этой статье мы рассмотрим ключевые аспекты аппаратного обеспечения и шаги, необходимые для развертывания серверов баз данных.

Требования к аппаратному обеспечению

Серверы баз данных имеют высокие требования к аппаратному обеспечению, чтобы справляться с интенсивной обработкой запросов и хранением больших объемов данных. Ниже представлены основные компоненты, которые необходимо учитывать:

1. Процессор (CPU)

Процессор является критически важным компонентом, влияющим на скорость обработки запросов. Серверы баз данных могут использовать многоядерные процессоры или даже несколько процессоров для параллельной обработки запросов. Количество ядер и тактовая частота процессора влияют на общую производительность сервера.

2. Оперативная память (RAM)

Оперативная память используется для хранения данных, которые часто запрашиваются, и для выполнения операций с базой данных. Больше памяти позволяет серверу кэшировать больше данных, что улучшает производительность, особенно при работе с большими таблицами. Обычно для серверов баз данных рекомендуется не менее 16 ГБ памяти, но для больших баз данных может потребоваться и больше.

3. Хранилище данных (Storage)

Серверы баз данных требуют надежного и быстрого хранилища для хранения

больших объемов данных. Это может включать в себя жесткие диски (HDD), твердотельные накопители (SSD) или сетевые хранилища (NAS). Скорость и емкость хранилища влияют на производительность ввода-вывода, что критично для баз данных с высокой нагрузкой.

4. Сетевое оборудование (Network)

Сетевое оборудование, такое как сетевые карты и коммутаторы, влияет на скорость передачи данных между сервером и клиентами. Высокоскоростные сетевые интерфейсы, такие как Ethernet 10Gbps или выше, могут улучшить производительность, особенно в распределенных системах.

					УП.09.02.07.22 ПЗ	Лист
						54
Изм.	Лист	№ докум.	Подпись	Дата		

23.РАССМОТРЕНИЕ СОСТАВА И СХЕМЫ БАНКА ДАННЫХ

Банк данных является важной частью систем управления базами данных (СУБД), обеспечивающей хранение и управление большими объемами данных. Он представляет собой структурированное хранилище, состоящее из нескольких компонентов, которые работают вместе для эффективного хранения и извлечения данных. В этой статье мы рассмотрим состав и схему банка данных, а также его роль в работе с базами данных.

Состав банка данных

Банк данных состоит из нескольких ключевых компонентов:

1. Диски (Disks)

Диски являются физическим хранилищем, на котором хранятся данные. Они могут быть жесткими дисками (HDD), твердотельными накопителями (SSD) или другими типами дисков. Диски обеспечивают долговечное и надежное хранение больших объемов данных.

2. Контроллер диска (Disk Controller)

Контроллер диска - это устройство, которое управляет доступом к дискам и обеспечивает передачу данных между дисками и сервером. Он отвечает за чтение и запись данных на диски, а также за управление кэшированием и буферизацией.

3. Буферный пул (Buffer Pool)

Буферный пул - это область памяти, используемая для кэширования данных с диска. Когда данные запрашиваются, система сначала проверяет буферный пул. Если данные уже находятся в буфере, они могут быть немедленно возвращены, что значительно ускоряет доступ.

24. РАЗРАБОТКА ТЕХНИЧЕСКИХ ТРЕБОВАНИЙ К СЕРВЕРУ БАЗ ДАННЫХ

24.1 РАЗРАБОТКА ТЕХНИЧЕСКИХ ТРЕБОВАНИЙ К СЕРВЕРУ БАЗ ДАННЫХ, ПРИМЕНИТЕЛЬНО К РАЗРАБАТЫВАЕМОМУ ЗАДАНИЮ

Разработка технических требований к серверу баз данных является важной задачей при создании системы управления базами данных. Эти требования определяют характеристики и возможности сервера, необходимые для эффективной работы с базой данных. Ниже представлены технические требования для сервера баз данных, предназначенного для хранения и управления учебной базой данных, описанной в предыдущих разделах.

1. Аппаратные требования

Сервер баз данных должен иметь следующие аппаратные характеристики:

Процессор: Многоядерный процессор с тактовой частотой не менее 2.0 ГГц. Рекомендуется использовать процессоры с поддержкой виртуализации и расширениями для работы с базами данных, такие как Intel Xeon или AMD EPYC.

Оперативная память: Не менее 32 ГБ памяти. Для больших баз данных и интенсивной нагрузки может потребоваться 64 ГБ или больше.

Хранилище данных: Не менее 1 ТБ дискового пространства. Рекомендуется использовать высокоскоростные диски, такие как SSD, для улучшения производительности ввода-вывода. Для обеспечения избыточности и надежности можно использовать RAID-массивы.

Сетевое оборудование: Высокоскоростные сетевые интерфейсы (10Gbps или выше) для обеспечения быстрой передачи данных между сервером и клиентами.

2. Требования к операционной системе

Сервер баз данных должен работать под управлением стабильной системы:

					УП.2023.09.02.07.ПЗ	Лист
						56
Изм.	Лист	№ докум.	Подпись	Дата		

Linux: Популярная операционная система для серверов баз данных, предлагающая широкий выбор дистрибутивов, таких как Ubuntu Server, CentOS, или Red Hat Enterprise Linux.

Windows Server: Предоставляет надежную и безопасную среду для серверов баз данных, особенно в экосистеме Microsoft.

3. Требования к системе управления базами данных (СУБД)

Сервер должен поддерживать одну из следующих систем управления базами данных:

MySQL: Популярная открытая СУБД, предлагающая высокую производительность и широкую поддержку различных платформ.

PostgreSQL: Мощная и гибкая СУБД с поддержкой расширенных функций и безопасности.

Microsoft SQL Server: Предлагает комплексное решение для управления базами данных, особенно в экосистеме Windows.

					УП.09.02.07.24 ПЗ	Лист
						57
Изм.	Лист	№ докум.	Подпись	Дата		

24.2 ТЕХНИЧЕСКИЕ ТРЕБОВАНИЯ К ПРОГРАММНЫМ СРЕДСТВАМ

- Операционная система (OS): Любая современная серверная операционная система, поддерживающая PHP и СУБД (например, MySQL/MariaDB). Наиболее распространенные варианты:
 - Linux: (Ubuntu, Debian, CentOS, Fedora) - предпочтительный выбор из-за стабильности, безопасности, гибкости и широкой поддержки.
 - Windows Server: Подходит, если есть опыт работы с Windows-серверами или требуется интеграция с существующей Windows-инфраструктурой.
- Веб-сервер: Для обработки HTTP-запросов и предоставления доступа к PHP-скриптам:
 - Apache HTTP Server: Один из самых популярных и надежных веб-серверов. Хорошо интегрируется с PHP и MySQL.
 - Nginx: Высокопроизводительный веб-сервер, который может использоваться как самостоятельно, так и в качестве прокси-сервера перед Apache.
- СУБД (Система управления базами данных): Для хранения и управления данными проекта:
 - MySQL: Открытая и широко используемая СУБД. Хорошо подходит для веб-приложений и имеет хорошую документацию.
 - MariaDB: Форк MySQL, разработанный сообществом. Часто используется в
 - качестве замены MySQL и обладает хорошей совместимостью.
 - Язык программирования:
 - PHP: (версии 7.4 или выше, рекомендуется 8.x) - для выполнения серверной логики, обработки запросов к базе данных и генерации HTML-кода.
 - Дополнительные библиотеки и расширения PHP:

- PDO (PHP Data Objects): Расширение для доступа к базам данных. Обеспечивает абстракцию от конкретной СУБД, что упрощает перенос приложения на другую СУБД.
- MySQLi: Альтернативное расширение для работы с MySQL.
- GD Graphics Library: Для обработки изображений (если потребуется).

25. РАЗРАБОТКА ТРЕБОВАНИЙ К КОРПОРАТИВНОЙ СЕТИ

25.1 РАЗРАБОТКА ТРЕБОВАНИЙ К КОРПОРАТИВНОЙ СЕТИ, ПРИМЕНИТЕЛЬНО К РАЗРАБАТЫВАЕМОМУ ЗАДАНИЮ

Корпоративная сеть играет важную роль в обеспечении доступа к учебной базе данных, позволяя пользователям и администраторам взаимодействовать с базой данных и обмениваться данными. Ниже представлены требования к корпоративной сети для эффективной работы с учебной базой данных.

1. Скорость и пропускная способность

Сеть должна обеспечивать высокую скорость и пропускную способность для передачи данных между сервером базы данных и клиентами. Рекомендуемая скорость не менее 1 Гбит/с (1000 Мбит/с) для локальной сети и не менее 100 Мбит/с для удаленных подключений. Это позволит быстро передавать запросы и получать результаты, особенно при работе с большими объемами данных.

2. Надежность и доступность

Корпоративная сеть должна быть надежной и доступной, обеспечивая бесперебойный доступ к базе данных. Это включает в себя использование избыточного оборудования, таких как резервные маршрутизаторы и коммутаторы, а также реализацию механизмов восстановления после сбоев, таких как резервное копирование и репликация данных.

3. Безопасность

Безопасность данных и сети является критически важным аспектом. Требования к безопасности включают:

Шифрование: Все данные, передаваемые по сети, должны быть зашифрованы для защиты от несанкционированного доступа. Это может быть реализовано с помощью протоколов, таких как SSL/TLS.

					УП.2023.09.02.07.ПЗ	Лист
						60
Изм.	Лист	№ докум.	Подпись	Дата		

25.2 ТРЕБОВАНИЯ К ЛОКАЛЬНОЙ СЕТИ ДЛЯ ФУНКЦИОНИРОВАНИЯ ВАШЕГО ИНТЕРФЕЙСА И БАЗЫ ДАННЫХ

Требования к локальной сети для функционирования административного интерфейса и базы данных, которые мы обсуждали, зависят от того, как именно развернуто приложение. Есть несколько распространенных сценариев развертывания, и для каждого из них будут свои требования.

Общие требования (для всех сценариев):

- ТСП/IP: Локальная сеть должна поддерживать протокол ТСП/IP. Это базовый протокол для любой современной сети и интернета, и он необходим для связи между веб-сервером и клиентами (браузерами) и сервером базы данных.

- Рабочая группа или домен: Компьютеры в сети должны быть настроены либо в одной рабочей группе, либо в одном домене. Это обеспечивает возможность обнаружения и связи между компьютерами.

- Разрешение имен (DNS или HOSTS): Компьютеры в сети должны уметь разрешать имена хостов в IP-адреса. Это может быть достигнуто с помощью DNS-сервера в сети или с помощью файлов hosts на каждом компьютере. Это необходимо для того, чтобы веб-сервер и сервер базы данных могли найти друг друга по имени хоста.

- Надежность: Сеть должна быть достаточно надежной, чтобы обеспечивать стабильную связь между веб-сервером, сервером базы данных и клиентами. Это включает в себя использование качественного сетевого оборудования (маршрутизаторы, коммутаторы, кабели) и наличие плана резервирования на случай сбоев.

Сценарии развертывания и специфические требования:

Все компоненты на одном сервере (самый простой случай):

- В этом сценарии веб-сервер (например, Apache или Nginx), сервер базы данных (например, MySQL или MariaDB) и все необходимые файлы (PHP-скрипты, HTML, CSS, JavaScript) находятся на одном и том же компьютере.

- Требования к сети:

					УП.09.02.07.25 ПЗ	Лист
						61
Изм.	Лист	№ докум.	Подпись	Дата		

- По сути, сетевое взаимодействие происходит внутри одного компьютера, поэтому требования к физической локальной сети минимальны.

- Нужен только TCP/IP.

- Но! Компьютер, на котором развернуто приложение, должен быть подключен

к локальной сети, чтобы к нему можно было получить доступ с других компьютеров.

- Конфигурация: В конфигурации веб-сервера нужно указать, чтобы он прослушивал все доступные сетевые интерфейсы (обычно это делается, указав адрес 0.0.0.0 или просто не указывая конкретный адрес). В конфигурации базы данных нужно указать, чтобы она принимала соединения с локального хоста (localhost или 127.0.0.1).

Веб-сервер и сервер базы данных на разных компьютерах:

- В этом сценарии веб-сервер и сервер базы данных находятся на разных компьютерах в локальной сети. Это более распространенный сценарий, так как он позволяет распределить нагрузку и повысить безопасность.

- Требования к сети:

- TCP/IP.

- Надежная связь между веб-сервером и сервером базы данных.

- Низкая задержка (latency) между веб-сервером и сервером базы данных (желательно).

26.РЕШЕНИЕ ВОПРОСОВ, СВЯЗАННЫХ С КОНФИГУРИРОВАНИЕМ СЕТИ

26.1 РЕШЕНИЕ ВОПРОСОВ, СВЯЗАННЫХ С КОНФИГУРИРОВАНИЕМ СЕТИ, ПРИМЕНИТЕЛЬНО К РАЗРАБАТЫВАЕМОМУ ЗАДАНИЮ

Конфигурирование сети является важной задачей для обеспечения эффективного взаимодействия между клиентами, сервером базы данных и другими компонентами в учебной среде. Ниже представлены шаги и рекомендации по конфигурированию сети для учебной базы данных.

1. Планирование топологии сети

Определите топологию сети, которая будет соответствовать требованиям учебной базы данных. Обычно это может быть одна из следующих конфигураций:

Локальная сеть (LAN): Все компоненты, включая сервер базы данных, клиентские компьютеры и другие устройства, находятся в одной локальной сети.

Распределенная сеть: Сервер базы данных может находиться в центре обработки данных или в облаке, а клиентские компьютеры подключаются через Интернет.

2. Настройка IP-адресов

Назначьте статические или динамические IP-адреса всем устройствам в сети. Это включает в себя сервер базы данных, клиентские компьютеры, маршрутизаторы и коммутаторы. Использование статических IP-адресов может упростить управление и обеспечение доступа к серверу базы данных.

3. Настройка маршрутизации

Настройте маршрутизацию, чтобы обеспечить связь между различными сегментами сети. Это особенно важно в распределенной сети, где сервер базы данных может находиться в другом сегменте или подсети.

26.2 КОНФИГУРАЦИЯ СЕТЕВОГО ОКРУЖЕНИЯ

Чтобы административный интерфейс работал корректно, необходимо

					УП.2023.09.02.07.ПЗ	Лист
						63
Изм.	Лист	№ докум.	Подпись	Дата		

настроить сетевое окружение. Вот описание требуемой конфигурации, основанное на предоставленном коде и предполагаемой функциональности:

1. Общая архитектура:

- Серверная часть: Веб-сервер (например, Apache или Nginx) с поддержкой PHP. Этот сервер будет размещать файлы интерфейса (PHP, CSS, JavaScript, изображения, звуки) и взаимодействовать с сервером базы данных.
- База данных: СУБД (например, MySQL или MariaDB), содержащая базу данных `project_Dimitrinev`.
- Клиентская часть: Веб-браузер пользователя, с которого осуществляется доступ к административному интерфейсу.

2. Требования к серверу:

- Операционная система: Любая ОС, поддерживающая веб-сервер и СУБД (например, Linux, Windows, macOS).
- Веб-сервер: Apache или Nginx. Рекомендуются Apache, так как это наиболее распространенный вариант для PHP.
- Конфигурация Apache:
- Поддержка PHP должна быть включена. Обычно это делается с помощью модуля `mod_php`.
- Необходимо настроить виртуальный хост для директории, где размещены файлы интерфейса. Укажите `DocumentRoot` и `DirectoryIndex` (например, `index.php`).
- Убедитесь, что директива `AllowOverride All` разрешена в блоке `<Directory>` для директории с файлами интерфейса, чтобы можно было использовать файлы `.htaccess`.
- PHP:
- Версия PHP 7.0 или выше (рекомендуется PHP 7.4 или 8.x).
- Необходимые расширения PHP:
- `mysqli` (или `pdo_mysql`): Для работы с базой данных MySQL/MariaDB.

27.РАЗРАБОТКА ХРАНИМЫХ ПРОЦЕДУР

27.1 РАЗРАБОТКА ХРАНИМЫХ ПРОЦЕДУР, ПРИМЕНительно К СОЗДАВАЕМОЙ УЧЕБНОЙ БАЗЕ ДАННЫХ

Хранимые процедуры являются мощным инструментом в системах управления базами данных, позволяющим автоматизировать задачи и улучшать производительность. В этой статье мы рассмотрим создание и использование хранимых процедур в контексте учебной базы данных, описанной в предыдущих разделах.

1. Создание процедуры для добавления десерта

Следующая хранимая процедура позволяет администратору добавлять новые десерты в базу данных:

```
1  DELIMITER //
2
3  CREATE PROCEDURE AddDessert(
4      IN p_Название VARCHAR(255),
5      IN p_Описание TEXT,
6      IN p_Цена DECIMAL(10,2),
7      IN p_Количество INT,
8      IN p_CategoryID INT
9  )
10 BEGIN
11     -- Проверка существования категории (опционально)
12     IF NOT EXISTS (SELECT 1 FROM Categories WHERE CategoryID = p_CategoryID) THEN
13         SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Указанная категория не существует';
14     END IF;
15
16     -- Вставка нового десерта
17     INSERT INTO Desserts (Название, Описание, Цена, Количество_на_складе, CategoryID)
18     VALUES (p_Название, p_Описание, p_Цена, p_Количество, p_CategoryID);
19
20     -- Возврат идентификатора созданной записи
21     SELECT LAST_INSERT_ID() AS NewDessertID;
22 END //
23
24 DELIMITER ;
```

Рисунок 7. Процедура 1

Эта процедура принимает параметры, такие как название, описание, цена, количество на складе и идентификатор категории, проверяет существование указанной категории, а затем вставляет новый десерт в таблицу «Десерты», возвращая идентификатор созданной записи.

27.2 ПРОЦЕДУРЫ РАБОТЫ С БАЗОЙ ДАННЫХ

1. Добавление функциональности на существующие страницы (CRUD):

- Общая структура: Для каждой таблицы (Материалы, Клиенты, Заказы и т.д.) нужно создать PHP-страницу, которая обеспечивает возможность выполнения основных операций CRUD (Create, Read, Update, Delete).

- Отображение данных (Read):

- Используйте SQL-запрос `SELECT * FROM <имя_таблицы>` для получения всех данных из таблицы.

- Отобразите данные в виде HTML-таблицы (`<table>`).

- Добавьте столбцы с кнопками “Редактировать” и “Удалить” для каждой строки.

- Добавление данных (Create):

- Создайте HTML-форму (`<form>`) с полями для ввода данных для каждого столбца таблицы.

- Используйте метод POST для отправки данных формы.

- На сервере, в PHP-скрипте, получите данные из `$_POST`.

- Выполните SQL-запрос `INSERT INTO <имя_таблицы> (<список_столбцов>) VALUES (<список_значений>)`. Обязательно используйте подготовленные запросы (prepared statements) для защиты от SQL-инъекций.

- После успешного добавления данных перенаправьте пользователя обратно на страницу просмотра таблицы.

- Редактирование данных (Update):

- При нажатии на кнопку “Редактировать” для строки, получите ID записи, которую нужно отредактировать.

- Выполните SQL-запрос `SELECT * FROM <имя_таблицы> WHERE <id_столбец> = <id_записи>` для получения данных записи.

- Заполните HTML-форму данными из базы данных.

- После изменения данных и нажатия кнопки “Сохранить”, получите данные из `$_POST`.

- Выполните SQL-запрос UPDATE <имя_таблицы> SET <список_столбцов_и_значений> WHERE <id_столбец> = <id_записи>. Используйте подготовленные запросы.
- Перенаправьте пользователя обратно на страницу просмотра таблицы.
- Удаление данных (Delete):
- При нажатии на кнопку “Удалить” для строки, получите ID записи, которую нужно удалить.
- Выполните SQL-запрос DELETE FROM <имя_таблицы> WHERE <id_столбец> = <id_записи>. Используйте подготовленные запросы.
- Перенаправьте пользователя обратно на страницу просмотра таблицы.

Рисунок 8. Процедуры

28. РАЗРАБОТКА ТРИГГЕРОВ БАЗЫ ДАННЫХ

28.1 РАЗРАБОТКА ТРИГГЕРОВ, ПРИМЕНИТЕЛЬНО К СОЗДАВАЕМОЙ УЧЕБНОЙ БАЗЕ ДАННЫХ

Триггеры являются мощным инструментом в системах управления базами данных, позволяющим автоматизировать задачи и обеспечивать целостность данных. В этой статье мы рассмотрим создание и использование триггеров в контексте учебной базы данных, описанной в предыдущих разделах.

1. Триггер для аудита изменений книг

Следующий триггер регистрирует все изменения в таблице "Диссерты", записывая старые и новые значения в таблицу аудита:

```
DELIMITER //
```

```
CREATE TRIGGER trg_Desserts_Audit_Update  
AFTER UPDATE ON Desserts  
FOR EACH ROW  
BEGIN  
    INSERT INTO Desserts_Audit (  
        DessertID,  
        Old_Название,  
        New_Название,  
        Old_Цена,  
        New_Цена,  
        Old_Количество,  
        New_Количество  
    )  
    VALUES (  
        OLD.DessertID,  
        OLD.Название,  
        NEW.Название,  
        OLD.Цена,  
        NEW.Цена,  
        OLD.Количество_на_складе,  
        NEW.Количество_на_складе  
    );  
END //
```

```
DELIMITER ;
```

Рисунок 9. Триггер 1

Этот триггер срабатывает после каждого обновления записи в таблице «Десерты» и записывает в таблицу аудита старые и новые значения для названия, цены и количества на складе, а также идентификатор изменённого десерта и время изменения.

28.2 РАЗРАБОТКА ТРИГГЕРОВ

Триггер - это автоматически выполняемая процедура в базе данных, срабатывающая при определенном событии (например, INSERT, UPDATE или DELETE) в указанной таблице. Триггеры полезны для обеспечения целостности данных, автоматизации задач и реализации сложной бизнес-логики.

1. Сценарии для триггеров

- Аудит изменений: Записывать изменения в таблицах в отдельную таблицу аудита (кто изменил, что изменил, когда).
 - Обеспечение целостности данных: Проверять данные при вставке или обновлении, чтобы убедиться, что они соответствуют определенным правилам (например, цена материала не может быть отрицательной).
 - Автоматическое обновление связанных таблиц: Обновлять данные в.
- Пример триггера (аудит изменений в таблице “Materials”)**

Предположим, мы хотим создать триггер, который записывает все изменения в таблице Materials в отдельную таблицу Materials_Audit.

```
-- Создаем таблицу для аудита
CREATE TABLE Materials_Audit
(   id_audit INT AUTO_INCREMENT PRIMARY KEY,   id_material INT
NOT NULL,   material_name VARCHAR(255),   material_price DECIMAL(10,2),
material_description TEXT,   date_changed DATETIME NOT NULL,
user_changed VARCHAR(50) -- Или id пользователя );
-- Создаем триггер AFTER UPDATE
CREATE TRIGGER Materials_After_Update
AFTER UPDATE ON Materials FOR EACH ROW
BEGIN   -- Вставляем запись в таблицу аудита
INSERT INTO Materials_Audit (   id_material,
material_name,   material_price,   material_description,
date_changed,   user_changed ) VALUES (   OLD.id_material,
OLD.material_name,   OLD.material_price,   OLD.material_description,
```

NOW(), 'AdminUser' -- Или получаем имя пользователя из сессии);
END;

29. РАССМОТРЕНИЕ ВОПРОСОВ, СВЯЗАННЫХ С ТЕХНОЛОГИЕЙ УСТАНОВКИ И НАСТРОЙКА СЕРВЕРА MYSQL В ОПЕРАЦИОННОЙ СИСТЕМЕ WINDOWS. КЛИЕНТСКИЕ НАСТРОЙКИ, ПРОТОКОЛИРОВАНИЕ, БЕЗОПАСНОСТЬ

MySQL является одной из самых популярных систем управления реляционными базами данных (РБД) и широко используется в различных приложениях. В этой статье мы рассмотрим процесс установки и базовой настройки сервера MySQL в операционной системе Windows, а также обсудим клиентские настройки, протоколирование и безопасность.

Установка MySQL

Чтобы установить MySQL на Windows, выполните следующие шаги:

Скачайте установщик: Посетите официальный сайт MySQL (<https://www.mysql.com>) и скачайте установщик для Windows. Выберите версию, соответствующую вашей операционной системе (32-бит или 64-бит).

Запустите установщик: Дважды щелкните по скачанному файлу установщика и следуйте инструкциям на экране. Выберите тип установки (например, "Developer Default" для разработки) и укажите путь установки.

Настройка параметров: Во время установки вы можете настроить параметры MySQL, такие как пароль администратора (root) и порт, на котором будет работать сервер. Запомните или запишите эти параметры, так как они будут необходимы для подключения к серверу.

Завершение установки: После завершения установки сервер MySQL будет запущен автоматически. Вы можете проверить его работу, открыв командную строку и выполнив команду `net start mysql` (или `systemctl start mysql` в более новых версиях Windows).

30. РАССМОТРЕНИЕ ВОПРОСОВ, СВЯЗАННЫХ С ТЕХНОЛОГИЕЙ УСТАНОВКИ СЕРВЕРА

30.1 РАССМОТРЕНИЕ ВОПРОСОВ, СВЯЗАННЫХ С ТЕХНОЛОГИЕЙ УСТАНОВКИ И НАСТРОЙКА СЕРВЕРА MYSQL В ОПЕРАЦИОННЫХ СИСТЕМАХ LINUX

MySQL является популярной системой управления реляционными базами данных (РБД) и широко используется в различных приложениях. В этой статье мы рассмотрим процесс установки и базовой настройки сервера MySQL в операционных системах Linux, а также обсудим некоторые важные аспекты конфигурирования и безопасности.

Установка MySQL

Установка MySQL в Linux может немного отличаться в зависимости от используемой дистрибутивной системы, но общие шаги остаются схожими. Ниже приведен процесс установки MySQL в Ubuntu Linux:

Обновление пакетов: Перед установкой MySQL убедитесь, что ваша система обновлена. Откройте терминал и выполните команду:

```
sudo apt update
```

Установка MySQL: Выполните команду для установки MySQL:

```
sudo apt install mysql-server
```

Эта команда установит сервер MySQL, клиентские библиотеки и утилиты.

Запуск и проверка сервера: После установки сервер MySQL должен запускаться автоматически. Вы можете проверить его состояние с помощью команды:

```
systemctl status mysql
```

Если сервер не запущен, вы можете запустить его вручную:

```
systemctl start mysql
```

30.2 ПРОЦЕСС УСТАНОВКИ И НАСТРОЙКИ ВАШЕЙ СЕРВЕРНОЙ OS И ВАШЕЙ БАЗЫ ДАННЫХ

A. Установка и настройка Ubuntu Server

1. Получение образа Ubuntu Server:

- Перейдите на официальный сайт Ubuntu: <https://ubuntu.com/download/server>
- Скачайте последний стабильный образ Ubuntu Server LTS (Long Term Support). LTS-версии поддерживаются дольше, что важно для стабильности сервера.

1. Создание загрузочного носителя:

- Вам потребуется USB-накопитель (минимум 4 ГБ) или DVD-диск.
- Используйте программу для записи образа на носитель. Для Windows можно использовать Rufus (<https://rufus.ie/en/>). Для Linux можно использовать dd или Etcher.

1. Установка Ubuntu Server:

- Загрузите компьютер с созданного носителя. Возможно, потребуется изменить порядок загрузки в BIOS/UEFI.
- Следуйте инструкциям на экране. Основные этапы:
- Выбор языка: Выберите язык интерфейса установки.
- Выбор раскладки клавиатуры: Выберите раскладку клавиатуры.
- Тип установки: Выберите “Ubuntu Server”.
- Сетевые настройки:
- DHCP (рекомендуется для начала): Если в сети есть DHCP-сервер, сервер получит IP-адрес автоматически.
- Ручная настройка (статический IP-адрес): Если требуется статический IP-адрес, укажите IP-адрес, маску подсети, шлюз и DNS-серверы. Статический IP-адрес может упростить доступ к серверу.
- Настройка диска:
- Использовать весь диск: Самый простой вариант. Выберите диск, на который будет установлена Ubuntu. Внимание: все данные на диске будут удалены!

31. РАССМОТРЕНИЕ ВОПРОСОВ АДМИНИСТРИРОВАНИЯ

31.1 РАССМОТРЕНИЕ ВОПРОСОВ, СВЯЗАННЫХ С УДАЛЕННЫМ АДМИНИСТРИРОВАНИЕМ

Удаленное администрирование MySQL позволяет администраторам управлять базами данных и серверами MySQL, расположенными на удаленных серверах или виртуальных машинах. Это особенно полезно в ситуациях, когда серверы находятся в центрах обработки данных или в облачной среде. В этой статье мы рассмотрим инструменты и методы для удаленного администрирования MySQL.

1. SSH (Secure Shell)

SSH является безопасным протоколом, позволяющим устанавливать зашифрованное соединение между двумя компьютерами. Он широко используется для удаленного доступа к командной строке Linux. С помощью SSH вы можете подключиться к удаленному серверу MySQL и выполнять административные задачи, как если бы вы были локально подключены.

Чтобы подключиться к удаленному серверу MySQL через SSH, выполните следующие шаги:

Установите SSH-клиент: Если вы используете Windows, скачайте и установите SSH-клиент, такой как PuTTY. В Linux и macOS SSH-клиент обычно уже установлен.

Получите доступ к серверу: Получите доступ к удаленному серверу через SSH, используя его IP-адрес или доменное имя. Например, в PuTTY введите IP-адрес, выберите SSH в качестве типа соединения и нажмите "Открыть".

Введите учетные данные: При первом подключении вы увидите предупреждение о том, что сервер не опознан. Нажмите "Да" или "Yes", чтобы продолжить. Затем введите имя пользователя и пароль для входа в систему.

31.2 ПРОЦЕСС УДАЛЕННОЙ РАБОТЫ

Этот процесс предполагает, что у вас есть административный интерфейс (веб-приложение), размещенный на сервере, как описано в предыдущих ответах. У нас есть:

Сервер: С установленной операционной системой (например, Ubuntu Server), веб-сервером (Apache), базой данных (MySQL) и PHP.

- Административный интерфейс: Веб-приложение, написанное на PHP, которое позволяет управлять базой данных.

Процесс удаленной работы можно разбить на следующие этапы:

1. Подготовка:

- Получение доступа к серверу:
- Узнайте IP-адрес или доменное имя сервера.
- Узнайте имя пользователя и пароль для доступа к серверу по SSH (Secure Shell).

- Настройка сети:
- Убедитесь, что ваш компьютер имеет доступ к серверу по сети. Проверьте подключение к интернету или локальной сети.

- Убедитесь, что брандмауэр на вашем компьютере и на сервере не блокирует необходимые порты (обычно 22 для SSH, 80 для HTTP и 443 для HTTPS).

- Установка SSH-клиента:
- Для Windows: Установите PuTTY (<https://www.putty.org/>) или другой SSH-клиент.

- Для macOS и Linux: SSH-клиент обычно уже установлен в терминале.

2. Подключение к серверу по SSH:

- Откройте SSH-клиент: Запустите PuTTY или откройте терминал.
- Введите данные для подключения:
- Host Name (or IP address): Введите IP-адрес или доменное имя сервера.
- Port: Введите порт SSH (обычно 22).

					УП.09.02.07.31 ПЗ	Лист
						75
Изм.	Лист	№ докум.	Подпись	Дата		

- Connection type: Выберите SSH.
- Подключитесь: Нажмите “Open” (в PuTTY) или введите команду `ssh username@server_ip_address` в терминале.

- Авторизуйтесь: Введите имя пользователя и пароль.

3. Работа с сервером через SSH:

- Выполнение команд: После успешного подключения вы получите доступ к командной строке сервера. Вы можете выполнять любые команды, доступные вашему пользователю.

- Управление файлами: Вы можете использовать команды `cd`, `ls`, `mkdir`, `rm`, `cp`, `mv` для навигации по файловой системе, просмотра файлов, создания директорий, удаления, копирования и перемещения файлов.

- Редактирование файлов:

- Используйте текстовые редакторы, доступные в командной строке (например, `nano`, `vim`, `emacs`).

- Используйте программу для передачи файлов (например, FileZilla или WinSCP) для загрузки файлов с сервера, редактирования их на своем компьютере и последующей загрузки обратно на сервер.

- Управление службами: Используйте команду `systemctl` для управления службами (например, `systemctl restart apache2` для перезапуска веб-сервера).

4. Доступ к административному интерфейсу через веб-браузер:

- Откройте веб-браузер: Запустите Chrome, Firefox, Safari или другой браузер.

- Введите адрес в адресной строке: Введите IP-адрес или доменное имя сервера. Если веб-приложение находится в поддиректории, укажите ее (например, `http://yourdomain.com/admin`). Если используется HTTPS, начните адрес с `https://`.

- Авторизуйтесь: Если требуется, введите логин и пароль для доступа к административному интерфейсу.

- Работайте с интерфейсом: Используйте интерфейс для управления базой данных, редактирования контента, просмотра статистики и выполнения других административных задач.

					УП.09.02.07.31 ПЗ	Лист
						77
Изм.	Лист	№ докум.	Подпись	Дата		

32. РАССМОТРЕНИЕ ВОПРОСОВ, СВЯЗАННЫХ С АУДИТОМ БАЗЫ ДАННЫХ

32.1 РАССМОТРЕНИЕ ВОПРОСОВ, СВЯЗАННЫХ С АУДИТОМ БАЗЫ ДАННЫХ, АУДИТОРСКИМ ЖУРНА- ЛОМ. УСТАНОВКА ОПЦИЙ, ВКЛЮЧЕНИЕ И ОТКЛЮЧЕ- НИЕ АУДИТА. ОЧИСТКА И УМЕНЬШЕНИЕ РАЗМЕРОВ ЖУРНАЛА

Аудит базы данных — это процесс отслеживания и регистрации событий, происходящих в базе данных, с целью обеспечения безопасности, контроля доступа и анализа действий пользователей. Аудит позволяет администраторам выявлять подозрительные операции, проверять соответствие политикам безопасности и выполнять требования регуляторов. Для эффективного управления аудитом важно понимать, как настраивать, включать, отключать и очищать аудиторский журнал.

Настройка и управление аудитом

1. Установка опций аудита. В большинстве СУБД (например, Oracle, SQL Server, PostgreSQL) аудит настраивается через параметры конфигурации. Например, в Oracle можно включить аудит с помощью команды `AUDIT`, указав объекты и действия, которые нужно отслеживать. В SQL Server аудит настраивается через компонент SQL Server Audit, где создаются спецификации аудита для сервера и базы данных.

2. Включение и отключение аудита. Аудит можно включить или отключить в зависимости от потребностей. Например, в Oracle команда `AUDIT SELECT TABLE` включает аудит для операций `SELECT`, а команда `NOAUDIT SELECT TABLE` отключает его. В SQL Server для включения аудита используется команда `ALTER SERVER AUDIT [AuditName] WITH (STATE = ON)`, а для отключения — `WITH (STATE = OFF)`.

32.2 СОЗДАНИЕ В ИНТЕРФЕЙСЕ КНОПКИ, ПОЗВОЛЯЮЩЕЙ ПРОИЗВЕСТИ АУДИТ И ЛОГИРОВАНИЕ ДЕЙСТВИЙ

- Создание таблицы в базе данных для хранения логов.
- Внедрение логики записи в лог в PHP-скрипты, выполняющие действия в

базе данных (CRUD-операции, выполнение SQL-запросов и т.д.).

Интерфейс для просмотра логов (Frontend):

- Создание страницы для отображения логов из таблицы в базе данных.
- Реализация фильтрации и сортировки логов (по дате, пользователю, действию и т.д.).

Кнопка аудита (Frontend):

- Добавление кнопки или ссылки в интерфейс для доступа к странице просмотра логов.

1. Создание таблицы для логов

Создайте таблицу AuditLog в базе данных для хранения информации о действиях пользователей:

```
CREATE TABLE AuditLog (  id_log INT AUTO_INCREMENT PRIMARY KEY,  timestamp DATETIME NOT NULL,  user VARCHAR(50) NOT NULL,  action VARCHAR(255) NOT NULL,  table_name VARCHAR(50),  record_id INT,  details TEXT,  ip_address VARCHAR(50) );
```

sql

- id_log: Уникальный идентификатор записи лога.
- timestamp: Дата и время события.
- user: Имя пользователя, выполнившего действие.
- action: Описание действия (например, “INSERT”, “UPDATE”, “DELETE”, “EXECUTE SQL”).
- table_name: Имя таблицы, затронутой действием.
- record_id: ID записи, затронутой действием (если применимо).

- details: Подробная информация о действии (например, измененные поля и их значения).

- ip_address: IP-адрес пользователя.

2. Внедрение логики логирования в PHP-скрипты

В каждом PHP-скрипте, выполняющем действия в базе данных, добавьте код для записи информации в таблицу AuditLog. Например, в materials.php, clients.php и process_sql.php.

```
<?php
function logAction(PDO $pdo, string $actionType, string $description, ?int $userId = null): void
{
    $sql = "INSERT INTO AuditLog (ActionType, Description, UserID, CreatedAt)
            VALUES (:actionType, :description, :userId, NOW())";
    $stmt = $pdo->prepare($sql);
    $stmt->execute([
        ':actionType' => $actionType,
        ':description' => $description,
        ':userId'      => $userId,
    ]);
}
```

Рисунок 10 . код

ЖУРНАЛ АУДИТА (AuditLog)				
ID	Тип действия	Описание	Пользоват	Дата и время
1	INSERT	Добавлен новый десерт «Торт Медовик»	42	2026-06-01 10:15
2	UPDATE	Изменена цена десерта ID=3 с 500 на 550	42	2026-06-01 10:20
3	DELETE	Удалён десерт ID=7	NULL	2026-06-01 10:25
4	LOGIN	Пользователь admin вошёл в систему	1	2026-06-01 10:30
5	INSERT	Добавлена категория «Чизкейки»	42	2026-06-01 10:35
6	UPDATE	Изменено количество десерта ID=2 с 10 на 20	42	2026-06-01 10:40
7	LOGOUT	Пользователь admin вышел из системы	1	2026-06-01 10:45
8	INSERT	Зарегистрирован новый клиент Иванов Иван	5	2026-06-01 10:50
9	DELETE	Удалён устаревший промокод «SUMMER2025»	42	2026-06-01 10:55
10	UPDATE	Обновлён статус заказа ID=101 на «Готов»	42	2026-06-01 11:00

Показаны последние 10 записей аудита. Используйте ↑↓ для прокрутки, Q для выхода.

Рисунок 11. Результат в действии

33. РАССМОТРЕНИЕ ВОПРОСОВ, СВЯЗАННЫХ С ТЕХНОЛОГИЯМИ СОЗДАНИЯ БАЗЫ ДАННЫХ

33.1 РАССМОТРЕНИЕ ВОПРОСОВ, СВЯЗАННЫХ С ТЕХНОЛОГИЯМИ СОЗДАНИЯ БАЗЫ ДАННЫХ С ПРИ- МЕНЕНИЕМ ЯЗЫКА SQL. ДОБАВЛЕНИЕ, УДАЛЕНИЕ ДАННЫХ И ТАБЛИЦ

Язык структурированных запросов (SQL) является мощным инструментом для управления реляционными базами данных. Он позволяет создавать, модифицировать и запрашивать данные эффективным и структурированным образом. В этой статье мы рассмотрим различные аспекты управления данными в базе данных с использованием языка SQL, включая добавление, изменение и удаление данных, а также создание и удаление таблиц.

Добавление данных

Чтобы добавить данные в таблицу, используется оператор INSERT. Он позволяет вставить одну или несколько строк в указанную таблицу. Ниже представлен пример добавления новой записи в таблицу "Клиенты":

```
INSERT INTO Customers (Name, Email, Phone)
VALUES ('John Doe', 'john@example.com', '123-456-7890');
```

В этом примере мы добавляем строку с именем "John Doe", электронной почтой "john@example.com" и номером телефона "123-456-7890" в таблицу "Клиенты". Структура таблицы должна соответствовать указанным столбцам.

Можно также добавить несколько строк за один запрос, используя оператор INSERT с предложением VALUES несколько раз:

```
INSERT INTO Customers (Name, Email, Phone)
VALUES ('Alice Smith', 'alice@example.com', '987-654-3210')
```

Изменение данных

Для изменения существующих данных в таблице используется оператор UPDATE. Он позволяет обновить значения в одной или нескольких строках. Например, чтобы изменить электронную почту клиента с именем "John Doe", можно использовать следующий запрос:

```
UPDATE Customers
```

					УП.2023.09.02.07.ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		81


```
SET Email = 'newemail@example.com'
```

```
WHERE Name = 'John Doe';
```

Этот запрос найдет строку с именем "John Doe" в таблице "Клиенты" и обновит значение электронной почты на "newemail@example.com". Предложение WHERE обеспечивает точное указание строки, которую нужно изменить.

Удаление данных

Удаление данных осуществляется с помощью оператора DELETE. Он позволяет удалить одну или несколько строк из таблицы. Например, чтобы удалить запись клиента с именем "Alice Smith", можно использовать следующий запрос:

```
DELETE FROM Customers
```

```
WHERE Name = 'Alice Smith';
```

Этот запрос найдет и удалит строку с именем "Alice Smith" из таблицы "Клиенты". Важно использовать условие WHERE, чтобы точно определить, какие строки необходимо удалить.

Создание таблиц

Для создания новой таблицы в базе данных используется оператор CREATE TABLE. Он определяет структуру таблицы, включая имена столбцов и их типы данных. Ниже представлен пример создания таблицы "Продукты":

```
CREATE TABLE Products (  
    ProductID INT PRIMARY KEY,  
    ProductName VARCHAR(255),  
    Price DECIMAL(10, 2)
```

33.2 ОПИСАНИЕ КОМАНД ДОБАВЛЕНИЯ, УДАЛЕНИЯ, ИЗМЕНЕНИЯ ДАННЫХ И ТАБЛИЦ

```
CREATE TABLE Products (  
    id_product INT AUTO_INCREMENT PRIMARY KEY,  
    product_name VARCHAR(255) NOT NULL,  
    product_price DECIMAL(10,2) NOT NULL,  
    product_description TEXT  
);
```

Удаление таблицы (DROP TABLE):

Синтаксис:

```
DROP TABLE table_name;
```

Описание: Удаляет указанную таблицу из базы данных (ВНИМАНИЕ: данные будут потеряны!).

table_name: Имя таблицы, которую нужно удалить.

Пример:

```
DROP TABLE Products;
```

Изменение таблицы (ALTER TABLE):

Синтаксис (добавление столбца):

```
ALTER TABLE table_name  
ADD column_name data_type constraints;
```

Синтаксис (удаление столбца):

```
ALTER TABLE table_name  
DROP COLUMN column_name;
```

Синтаксис (изменение типа данных столбца):

```
ALTER TABLE table_name  
MODIFY COLUMN column_name data_type constraints;
```

Описание: Изменяет структуру существующей таблицы.

table_name: Имя таблицы, которую нужно изменить.

ADD column_name data_type constraints: Добавляет новый столбец.

34. РАССМОТРЕНИЕ ВОПРОСОВ, СВЯЗАННЫХ С СОЗДАНИЕМ ЗАПРОСОВ, ПРОЦЕДУР И ТРИГГЕРОВ. СОЗДАНИЕ ЗАПРОСОВ И ПРОЦЕДУР НА ИЗМЕНЕНИЕ СТРУКТУРЫ БАЗЫ ДАННЫХ

Язык структурированных запросов (SQL) является мощным инструментом для управления реляционными базами данных. Он позволяет создавать, модифицировать и запрашивать данные эффективным и структурированным образом. В этой статье мы рассмотрим различные аспекты управления данными в базе данных с использованием языка SQL, включая добавление, изменение и удаление данных, а также создание и удаление таблиц.

Добавление данных

Чтобы добавить данные в таблицу, используется оператор INSERT. Он позволяет вставить одну или несколько строк в указанную таблицу. Ниже представлен пример добавления новой записи в таблицу "Клиенты":

```
INSERT INTO Customers (Name, Email, Phone)
VALUES ('John Doe', 'john@example.com', '123-456-7890');
```

В этом примере мы добавляем строку с именем "John Doe", электронной почтой "john@example.com" и номером телефона "123-456-7890" в таблицу "Клиенты". Структура таблицы должна соответствовать указанным столбцам.

Можно также добавить несколько строк за один запрос, используя оператор INSERT с предложением VALUES несколько раз:

```
INSERT INTO Customers (Name, Email, Phone)
VALUES ('Alice Smith', 'alice@example.com', '987-654-3210'),
('Bob Johnson', 'bob@example.com', '555-123-4567');
```

Этот запрос добавит две новые записи в таблицу "Клиенты".

35. РАССМОТРЕНИЕ ВОПРОСОВ, СВЯЗАННЫХ С ДИНАМИЧЕСКИМ SQL И ЕГО ОПЕРАТОРАМИ

Динамический SQL является мощным инструментом в арсенале разработчиков баз данных, позволяющим создавать гибкие и адаптивные запросы. Он обеспечивает возможность генерировать SQL-код динамически во время выполнения, что открывает широкие возможности для решения сложных задач. В этой статье мы рассмотрим динамический SQL, его операторы и преимущества использования.

Что такое динамический SQL?

Динамический SQL - это подход, при котором SQL-запросы формируются и выполняются динамически, в зависимости от определенных условий или входных параметров. Он позволяет создавать более гибкие и адаптивные решения, особенно в ситуациях, когда структура запроса может варьироваться. Динамический SQL часто используется для создания общих процедур или функций, которые могут работать с различными наборами данных.

Операторы динамического SQL

EXECUTE IMMEDIATE:

Оператор EXECUTE IMMEDIATE позволяет выполнять динамические SQL-запросы. Он принимает строку, содержащую SQL-код, и выполняет этот код во время выполнения. Пример:

DECLARE

sql_query VARCHAR(200);

BEGIN

sql_query := 'SELECT * FROM Customers WHERE Country = "USA"';

EXECUTE IMMEDIATE sql_query;

END;

ЗАКЛЮЧЕНИЕ

В ходе учебной практики была проведена комплексная работа по изучению ключевых аспектов администрирования баз данных (БД), проектирования и реализации учебной базы данных, а также анализа современных технологий управления данными. Практика позволила закрепить теоретические знания и развить навыки, необходимые для профессиональной деятельности администратора БД.

Администрирование баз данных:

Рассмотрены обязанности администратора БД, включая управление пользователями, привилегиями, табличными пространствами, файлами данных и структурой памяти. Освоены режимы запуска/останова БД, работа с журналами и контрольными точками, что обеспечивает надёжность и целостность данных.

Изучены инструменты для управления транзакциями, блокировками и согласованностью данных, а также принципы работы с распределёнными БД и многопроцессорными архитектурами.

Проектирование учебной БД:

Разработана схема и словарь данных, определены модели и типы данных, созданы сценарии работы с данными.

Практически применены навыки написания команд администрирования, хранимых процедур, триггеров и динамического SQL, что позволило автоматизировать процессы изменения структуры БД и обработки данных.

Серверные и сетевые аспекты:

Проанализированы требования к серверному и клиентскому ПО, аппаратному обеспечению, корпоративным сетям и протоколам удалённого вызова.

Изучены технологии установки и настройки сервера MySQL в ОС.

Безопасность и оптимизация:

Освоены методы аудита БД, управления аудиторскими журналами, а также конфигурирования сети для обеспечения безопасности и производительности.

					УП.2023.09.02.07.ПЗ	Лист
						86
Изм.	Лист	№ докум.	Подпись	Дата		

Рассмотрены технологии оптимизации запросов, работы с динамическим SQL и операциями над структурой БД.

Итоги:

Практика позволила систематизировать знания в области проектирования,

администрирования и оптимизации баз данных. Реализация учебной БД продемонстрировала умение применять инструменты SQL, управлять объектами схемы, настраивать серверные среды и обеспечивать безопасность данных. Полученный опыт станет основой для дальнейшего профессионального роста в области управления данными и ИТ-инфраструктурой.

Результаты работы подтверждают готовность к решению задач, связанных с администрированием БД, проектированием информационных систем и поддержанием их стабильной работы в корпоративных средах.

					УП.2023.09.02.07.ПЗ	Лист
						87
Изм.	Лист	№ докум.	Подпись	Дата		

СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ

- 1.Перлова, О. Н. Сoadминистрирование баз данных и серверов : учебник для студентов среднего профессионального образования по специальности 09.02.07 "Информационные системы и программирование" / О. Н. Перлова, О. П. Ляпина ; О. Н. Перлова, О. П. Ляпина. – 2-е изд., испр. – Москва : Академия, 2020. – 304 с. – ISBN9785446894246. – URL:<https://academia-moscow.ru/catalogue/4831/480248/>. – Текст : электронный.
- 2.Федорова, Г. Н. Основы проектирования баз данных : учебник для образовательных учреждений, реализующих программы среднего профессионального образования по специальности "Информационные системы (по отраслям)" / Г. Н. Федорова ; Г. Н. Федорова. – 4-е изд., перераб. – Москва : Академия, 2020. – 224 с. – (Профессиональное образование : Информатика и вычислительная техника). – ISBN9785446886913. – URL:<https://academia-moscow.ru/catalogue/4831/468044/>. – Текст : электронный.
- 3.Проектирование информационных систем : учебник и практикум для среднего профессионального образования / Д. В. Чистов, П. П. Мельников, А. В. Золотарюк, Н. Б. Ничепорук. — 2-е изд., перераб. и доп. — Москва : Издательство Юрайт, 2024. — 293 с. — (Профессиональное образование). — ISBN 978-5-534-16217-2. — Текст : электронный // Образовательная платформа Юрайт [сайт]. — URL:<https://urait.ru/bcode/538370>.
- 4.Федорова, Г. Н. Разработка, внедрение и адаптация программного обеспечения отраслевой направленности : Учебное пособие / Г. Н. Федорова. – Москва : КУРС : ИНФРА-М, 2022. – 336 с. (Среднее профессиональное образование). – ISBN9785906818416. – Текст : непосредственный.
- 5.Компьютерные сети : учебник для среднего профессионального образования по специальностям 09.02.06 "Сетевое и системное администрирование", 09.02.07 "Информационные системы и программирование" / В. В. Баринов, И. В. Баринов, А. В. Пролетарский, А. Н. Пылькин ; В. В. Баринов [и др.]. – 4-е изд.,

					УП.2023.09.02.07.ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		88

испр. и доп.. - Москва : Академия, 2021. – 192 с. – ISBN9785446899258. – URL:<https://academia-moscow.ru/catalogue/4831/551458/>. – Текст : электронный.

6. Гохберг, Г. С. Информационные технологии : учебник для образовательных организаций, реализующих программы среднего профессионального образования по специальностям "Информационные системы и программирование", "Сетевое и системное администрирование" / Г. С. Гохберг, А. В. Зафиевский, А. А. Короткин ; Г. С. Гохберг, А. В. Зафиевский, А. А. Короткин. – 3-е изд., стер. - Москва : Академия, 2020. – 240 с. – (Профессиональное образование). – ISBN9785446886845. – URL:<https://academia-moscow.ru/catalogue/4831/471778/>. – Текст : электронный.

7. Казанский, А. А. Программирование на VisualC# : учебное пособие для среднего профессионального образования / А. А. Казанский. — 2-е изд., перераб. и доп. — Москва : Издательство Юрайт, 2024. — 192 с. — (Профессиональное образование). — ISBN 978-5-534-14130-6. — Текст : электронный // Образовательная платформа Юрайт [сайт]. — URL:<https://urait.ru/bcode/538155>.

8. Семакин, И. Г. Основы алгоритмизации и программирования : учебник для образовательных организаций, реализующих программы среднего профессионального образования по специальностям "Информационные системы и программирование", "Сетевое и системное администрирование", "Обеспечение информационной безопасности автоматизированных систем", "Обеспечение информационной / И. Г. Семакин, А. П. Шестаков ; И. Г. Семакин, А. П. Шестаков. – 4-е изд., стер. - Москва : Академия, 2020. – 304 с. – (Профессиональное образование). – ISBN99785446886883. – URL:<https://academia-moscow.ru/catalogue/4831/471483/>. – Текст : электронный.

9. Куприянов, Д. В. Информационное обеспечение профессиональной деятельности : учебник и практикум для среднего профессионального образования / Д. В. Куприянов. — 2-е изд., перераб. и доп. — Москва : Издательство Юрайт, 2024. — 283 с. — (Профессиональное образование). — ISBN

					УП.2023.09.02.07.ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		89

978-5-534-17829-6. — Текст : электронный // Образовательная платформа Юрайт [сайт]. — URL:<https://urait.ru/bcode/537693>.

10. Грекул, В. И. Проектирование информационных систем : учебник и практикум для среднего профессионального образования / В. И. Грекул, Н. Л. Коровкина, Г. А. Левочкина. — 2-е изд. — Москва : Издательство Юрайт, 2024. — 423 с. — (Профессиональное образование). — ISBN 978-5-534-17836-4. — Текст : электронный // Образовательная платформа Юрайт [сайт]. — URL:<https://urait.ru/bcode/543034>.
11. Производственная практика (преддипломная): методические материалы для обучающихся специальности СПО 09.02.07 "Информационные системы и программирование" очной формы обучения / ФГБОУ ВО «Кузбас. гос. техн. ун-т им. Т. Ф. Горбачева», Каф. информ. и автоматизир. произв. систем; сост. И. В. Чичерин. — Кемерово: КузГТУ, 2018. — 11 с. — URL: <http://library.kuzstu.ru/meto.php?n=9778>. — Текст: электронный.
12. Официальный сайт Кузбасского государственного технического университета имени Т.Ф. Горбачева. Режим доступа: <https://kuzstu.ru/>.
13. Официальный сайт филиала КузГТУ в г. Белово. Режим доступа: <http://belovokyzgty.ru>.
14. Электронная обучающая система филиала КузГТУ в г. Белово. Режим доступа: <http://eos.belovokyzgty.ru/>.
15. Электронные библиотечные системы:
- 16.- Образовательная платформа Юрайт [Электронный ресурс]. — Режим доступа:<https://urait.ru/>.
- 17.- Электронная библиотечная система «Лань» [Электронный ресурс]. — Режим доступа:<https://e.lanbook.com/>
- 18.- Электронная библиотека издательства Академия [Электронный ресурс]. — Режим доступа: <https://academia-library.ru/>
- 19.- Электронная библиотечная система ZNANIUM.COM [Электронный ресурс]. — Режим доступа:<https://znanium.com/>

- 20.Единая коллекция Цифровых образовательных ресурсов [Электронный ресурс]. – Режим доступа: <http://school-collection.edu.ru/>, свободный. – Загл. с экрана.
- 21.Единое окно доступа к информационным ресурсам [Электронный ресурс]. – Режим доступа:<http://window.edu.ru/>, свободный. – Загл. с экрана.
- 22.Всероссийский образовательный портал «ИКТ педагогам» [Электронный ресурс]. – Режим доступа:<https://edu-ikt.ru>, свободный. – Загл. с экрана.
- 23.Федеральный центр информационно-образовательных ресурсов [Электронный ресурс]. – Режим доступа: <http://fcior.edu.ru/>, свободный. – Загл. с экрана.